

附录 A: Platform Usage Tips (平台使用提示)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 全书唯一一个"不上课"的章节——它不教语言语法, 而是教你在 Windows、macOS、Linux、Android、iOS 五大平台上把 Python 装起来、跑起来。作者把它放在附录, 因为它与语言核心无关, 但又是你真正动手写代码前必须先解决的事。用作者的话说: keeping it simple (保持简单) ——尽量走最省事的路。

A.1 开篇引言 (Introduction)

英文原文

One of Python's main strengths is its portability: most of the code you'll write in your Python programs will work the same on all computing platforms. This has limits, of course; programs that use Python's portable libraries weather device hops better than others, and platform idiosyncrasies and restrictions can sometimes pose interoperability hurdles that require special handling. But by and large, the Python language is cross-platform by design. Python's portability means you can run its code on just about every computing dev

ice on the planet—from smartphones and tablets to PCs and supercomputers—and each of these systems has unique setup and usage details. While this appendix cannot be an exhaustive user guide for every one of those devices, it provides just enough info to help you prepare to run this book's code examples on your popular platform—or platforms—of choice, including Windows, macOS, Linux, Android, and iOS. Before we get started, here are two quick content notes up front. First, because you're going to have enough on your plate just learning Python itself, the focus throughout this appendix is on keeping it simple. There are many ways to run code, and you may find advanced options useful once you graduate from Python novice to master. Especially when starting out, though, this book recommends walking the easiest path. Second, a usage appendix like this is unavoidably doomed to grow out of date soon, given the rapid and constant change in the computing world (even the name of macOS, after all, has changed repeatedly on this book's watch!). Hence, please consider this a snapshot of the current state and practice, and plan to consult the latest resources if/when this story changes. For the present, let's jump right into today's usage options—while they last.

中文翻译

Python 的主要优势之一是它的可移植性 (portability) : 你在 Python 程序中写的大部分代码, 在所有计算平台上都会以相同方式工作。当然, 这是有限度的; 使用 Python 可移植库的程序比其他的更能经受住设备切换的考验, 而平台的怪癖和限制有时会造成需要特殊处理的互操作性障碍。但总体而言, Python 语言在设计上就是跨平台 (cross-platform) 的。Python 的可移植性意味着你几乎可以在地球上任何计算设备上运行它的代码——从智能手机、平板电脑到 PC 和超级计算机——而每套系统都有独特的安装设置和使用细节。虽然本附录不可能成为每一种设备的详尽用户指南, 但它提供了足够的信息, 帮助你准备好在你所选的流行平台——或几个平台——上运行本书的代码示例, 包括 Windows、macOS、Linux、Android 和 iOS。在开始之前, 先提前说明两点。第一, 你光是学习 Python 本身就已经够忙的了, 因此整个附录的重点是保持简单 (keeping it simple)。运行代码的方式有很多, 等你从 Python 新手进阶为大师之后, 可能会发现高级选项很有用。不过, 尤其是在起步阶段, 本书建议走最省事的路。第二, 考虑到计算世界日新月异 (毕竟, 就连 macOS 这个名字, 在这本书的"有生之年"里也已经反复改过!), 像这样的使用附录难免很快就会过时。因此, 请把它视为当前状态和实践的一张快照 (snapshot), 并计划在情况变化时查阅最新资料。就目前而言, 让我们赶紧进入今天的各种用法——趁着它们还没过期。

深度理解

·核心概念: 本附录教的是"环境搭建", 与语法无关。全书的知识体系里, 它是唯一一个"用不到大脑、只用到手"的

章节——但偏偏是它决定你能不能打开 Python 的大门。

·底层视角：可移植性为什么能成立？因为 CPython 把平台差异（文件路径、换行符、进程创建、命令行）全部封装进了解释器和标准库内部。你写 `os.path.join()`，解释器在 Windows 上给你拼 `\`，在 Unix 上给你拼 `/`——平台差异被"消化"在解释器这一层。

·设计思想：作者为什么坚持"保持简单"？因为新手的失败大多不是语法错误，而是"环境没跑通"。装 Python 这条路上岔路极多（商店版、官方版、WSL、Homebrew、源码编译……），每个都合法但都消耗注意力。学习期只走一条最短路，是性价比最高的策略。

·实际问题：这是 2023 年前后写作的"快照"，如今 macOS 默认不再预装 Python、Windows Store 版早已不是唯一入口、Android 上 CPython 已正式支持——信息一定会过期，所以作者反复提醒：以官方文档为准。

·初学者误区：以为"安装 Python"等于"双击安装包"这么简单。实际上各平台的安装都有隐藏岔路（Windows 的 PATH 勾选、macOS 的 Xcode 弹窗、Linux 的多发行版差异），这正是本附录要逐一拆解的。

A.2 在 Windows 上使用 Python (Using Python on Windows)

A.2.1 三种使用方式总览 (Windows, WSL, Cygwin)

英文原文

As the current market-share leader for PCs, Windows will undoubtedly play host to many readers' first encounter with Python. Python has been completely usable on this platform since its earliest days, and goes out of its way to smooth Windows' proprietary edges so your code doesn't have to. A well-coded Python program from Unix, for example, often runs unchanged on Windows despite the two platforms' many glaring differences. Today, there are at least three different ways to use Python on Windows: in Windows itself, in Windows Subsystem for Linux (a.k.a. WSL and WSL2), and in the third-party Cygwin Unix-like environment. We won't cover Cygwin here because it's not as widely used, and those who may care to use it probably already know how to use it.

中文翻译

作为 PC 市场占有率目前的领头羊，Windows 毫无疑问将是许多读者第一次接触 Python 的平台。Python 从诞生之初起就能在这个平台上完全可用，并且不遗余力地抹平 Windows 的专有棱角，让你的代码不必去迁就它们。例如，一个在 Unix 上写得很好的 Python 程序，尽管两大平台差异悬殊，往往也能在 Windows 上原样运行。如今，在 Wi

Windows 上使用 Python 至少有三种不同的方式：在 Windows 本身中运行；在适用于 Linux 的 Windows 子系统（即 WSL 和 WSL2）中运行；以及在第三方的类 Unix 环境 Cygwin 中运行。这里我们不讨论 Cygwin，因为它用得没那么广，而且想用的人大概早就知道怎么用了。

深度理解

- 核心概念：Windows 上有三条路：原生（native）、WSL（Linux 子系统）、Cygwin（Unix 模拟层）。本书推荐原生——“移动部件最少”。

- 底层视角：三者性质完全不同：原生 = 为 Windows 编译的 CPython 可执行文件；WSL = 微软提供的真正 Linux 内核（WSL2 是轻量虚拟机里跑完整内核）；Cygwin = 在 Windows 上翻译 POSIX 系统调用的兼容层。只有原生才称得上“零转换”。

- 设计思想：Python 从一开始就在 Windows 上可用，因为 Python 的设计目标就是“代码不该为平台弯腰”——标准库的 os、sys、shutil 把平台差异挡在身后。

- 实际问题：WSL 需要额外安装步骤、占用磁盘、并且“仍有接缝”（作者原话）；Cygwin 是另类小众。多数读者的最优解就是原生安装。

- 初学者误区：以为“用 WSL 才算专业”。恰恰相反——新手期用原生 Windows 版本最省事，等熟悉了再按需探索 WSL。

A.2.2 WSL (Windows Subsystem for Linux)

英文原文

WSL—including its newer WSL2 variant—brings Linux to your Windows PC without the hassles of dual-boot installs or separate devices. You get a standard Linux distribution (e.g., Ubuntu) that runs in the Windows UI and avoids some of the trade-offs of classic virtual machines. WSL comes with a command line to edit and run Python code, and WSL2 even runs Linux GUI apps (though they're still marginal at this writing). Because WSL is really Linux, we'll defer to the Linux section ahead for Python setup and usage info, as well as Microsoft's online documentation for details on installing WSL itself. While WSL may pique some readers' interest, it requires extra setup steps and is not wholly without seams today. For most readers, the easiest way to use Python on Windows is to go native—with a Python built to run in Windows directly.

中文翻译

WSL——包括它较新的 WSL2 变体——把 Linux 带到你的 Windows PC 上，免去了双系统（dual-boot）安装或额外设备的麻烦。你会得到一个运行在 Windows 界面里的标准 Linux 发行版（例如 Ubuntu），并且绕开了经典虚拟机（virtual machine）的某些折衷。WSL 自带一个命令行，

可以编辑和运行 Python 代码，WSL2 甚至能运行 Linux 图形界面应用（不过在本书写作时它们还相当边缘）。由于 WSL 本质上就是 Linux，Python 的安装与使用信息我们就留到后面的 Linux 一节去讲；安装 WSL 本身的细节请参考微软的在线文档。虽然 WSL 可能会勾起一些读者的兴趣，但它需要额外的安装步骤，而且今天也并非完全没有“接缝”。对大多数读者来说，在 Windows 上使用 Python 最简单的方式是原生（native）安装——装一个直接面向 Windows 编译的 Python。

深度理解

- 核心概念：WSL 不是模拟器，也不是“Windows 里的一个软件”——它是一个真正运行 Linux 的子系统。WSL2 在轻量虚拟机中运行完整 Linux 内核，性能接近裸机。
- 底层视角：经典虚拟机要虚拟化整台电脑（CPU、内存、外设），开销大；WSL2 只虚拟化 Linux 内核所需的最小集合，与 Windows 共享文件系统，启动只需几秒。
- 设计思想：作者特意强调“WSL 就是 Linux”——这意味着所有 Python 配置照 Linux 的规则来（apt 安装、#! 行、PATH），别再想 Windows 那套。
- 实际问题：WSL 的坑在于“额外步骤 + 仍有接缝”：文件在 Windows 与 WSL 之间穿梭的性能损耗、图形应用还不成熟。新手犯不上。
- 初学者误区：把 WSL 当成“Windows 里的 Python 完美方案”。它是给“想在 Windows 上过 Linux 生活”的开发者准备的，不是给新手的第一选择。

A.2.3 安装: python.org 与 Microsoft Store (Installation)

英文原文

Python doesn't come with Windows, but it is easy to install and use there. In short, Python for Windows can be installed by downloading a self-installer or visiting the Microsoft Store, and can be run with a simple Windows command-line interface; a graphical IDE—a GUI for editing and launching code; and clicks on program-file entries in Windows File Explorer. In more detail, the recommended way to install Python for Windows begins with a visit to the Downloads page at python.org. There, you'll fetch a Windows self-installer that you'll run to install Python 3.X, along with its IDLE GUI and standard library (including the library's tkinter GUI toolkit). Figure A-1 captures the Python installer in action; allow it to run if Windows asks for permission. You can generally accept all installation defaults, but it's recommended to opt in to both adding Python to your PATH at the start of the install and lifting the Windows path-length limits for filenames at the end. NOTE — Installation convolution: Though less common, you can also install Python from the Microsoft Store. In fact, typing `python3` in a Windows command line at this writing automatically routes you to the store to run the install—confusingly! If you opt to acc

apt this offer, python3 will run the store's version after the install, and the Start menu will sprout separate entries for launching this Python and its IDLE (up shortly). This guide uses and generally recommends the python.org install (and its py helper) because it's more traditional and may be better suited to general use. That said, the store version ultimately comes from the same source, and either may be used for this book's examples on Windows. But beware: the store version imposes access restrictions that may matter to you later and should probably relegate it to a secondary option for most readers. You really shouldn't install both the store and nonstore Windows Pythons, though, unless you need more drama in your life!

中文翻译

Windows 并不自带 Python，但安装和使用都很容易。简单来说，Windows 版 Python 可以通过下载自安装程序（self-installer）或访问 Microsoft Store 来安装；运行方式则包括：简单的 Windows 命令行界面；图形化 IDE——一个用于编辑和启动代码的 GUI；以及在 Windows 文件资源管理器中点击程序文件条目。更详细地说，推荐的 Windows 版 Python 安装方式，是从访问 python.org 的 Downloads 页面开始。在那里，你会下载一个 Windows 自安装程序，运行它即可安装 Python 3.X，同时装上它的 IDLE GUI 和标准库（包括库里的 tkinter GUI 工具包）。图 A-1 抓拍了正在运行的 Python 安装程序；如果 Windows 请求权限，请允许它运行。一般情况下你可以接受所有安装

默认值，但建议勾选两个选项：安装开始时把 Python 加入 PATH，以及安装结束时解除 Windows 的文件名路径长度限制。注意——安装的弯弯绕绕：虽然不太常见，你也可以从 Microsoft Store 安装 Python。事实上，在本书写作时，在 Windows 命令行里输入 `python3` 会自动把你带到商店去执行安装——真是让人困惑！如果你接受了这个提议，安装完成后 `python3` 将运行商店版，而开始菜单里也会冒出两个单独的条目，分别用于启动这个 Python 和它的 IDLE（稍后详述）。本指南使用并通常推荐 `python.org` 安装（以及它的 `py` 辅助命令），因为它更传统，可能更适合一般用途。话虽如此，商店版归根结底也来自同一个源头，两者都可以用来运行本书在 Windows 上的示例。但要当心：商店版会施加访问限制（access restrictions），这些限制以后可能会对你有影响，对大多数读者来说它应该退居次要选项。不过，你真的不该同时安装商店版和非商店版两个 Windows Python——除非你想让生活多点“戏”！

深度理解

·核心概念：Windows 有两条官方安装渠道：`python.org` 官网安装器（推荐）与 Microsoft Store 商店版。两者源码同源，但商店版有沙箱访问限制。

·底层视角：两个勾选选项背后是实打实的机制——PATH 决定命令行能否直接敲 `python/py`；Windows 旧默认路径上限 260 字符（MAXPATH），Python 的安装器可以帮你写入注册表开启长路径支持。

·设计思想：商店版的存在是为普通消费者提供“应用商店式”安装体验；但 Python 是开发工具，需要读写文件、装

扩展包，商店沙箱反而碍事——这正是“访问限制”的由来。

·实际问题：微软的“贴心”埋了个雷：命令行敲 `python3` 竟然会被劫持跳转到商店！这是 Windows 平台特有的坑，作者专门用 NOTE 标注。

·初学者误区：①以为 `python3` 在 Windows 上一定可用——只有商店版安装后才行；②一口气装了官网版又装商店版——两个 Python 并存会让你在调试时精神分裂；③不勾选 PATH，装完发现命令行找不到 `python`。

A.2.4 命令行运行与 py 启动器 (Command Lines and the py Launcher)

英文原文

Once you've installed Python on Windows, running it there can be as simple as typing code at a command line and clicking file icons or as complex as learning the nuances of a full-featured IDE. Of these, command lines generally add the least number of moving parts. To run Python from a command line on Windows, open either Command Prompt or PowerShell (the non-ISE flavor, normally). Once open, simply type `py` or other options presented in a moment and add a filename to run a file of code (i.e., a script). For example, open Command Prompt from your Start menu (search for it there if needed). Then, type `py` and press Enter

to start a Python interactive session where you can type and run code at the `>>>` prompt. To launch a script instead, type `py script.py`, with the name of your script. Figure A-2 demos these commands live on Windows. For space, some demos in this appendix, including this one, use Python's `-q` flag to suppress messages on session startup; this is cosmetic and optional. The `py` command is technically part of the Python Windows launcher that's installed along with Python itself. By default, the launcher runs the most recent Python version installed on your PC, but you can also specify a version to run if there's more than one (e.g., `py -3` runs the latest 3.X, and `py -3.8` runs an older version of it). If you have just one Python or want to use the latest, `py` suffices to launch an interactive session or script. You can also start Python with command `python` if you opted to add Python to your system PATH during the install, though it's pointless extra typing (`python3` works too, but only if you installed from the Store per the earlier note). And just as on Unix, you can easily save a script's printed output to a file by adding `> filename.txt` to the end of a command (see Chapter 3 for more on such stream redirections).

中文翻译

在 Windows 上装好 Python 之后，运行它可以简单到在命令行里输入代码、点击文件图标，也可以复杂到去琢磨一个全功能 IDE 的种种细节。其中，命令行通常引入的“移动部

件"最少。要在 Windows 命令行运行 Python，请打开命令提示符（Command Prompt）或 PowerShell（通常是非 ISE 版本）。打开后，直接输入 `py`，或稍后介绍的其他选项，再附上文件名即可运行代码文件（也就是脚本，`script`）。例如，从开始菜单打开命令提示符（需要的话可以在那里搜索它）。然后输入 `py` 并按回车，启动一个 Python 交互式会话（interactive session），你就可以在 `>>>` 提示符下输入并运行代码了。如果想改为启动脚本，就输入 `py script.py`，其中 `script.py` 是你脚本的名字。图 A-2 在 Windows 上现场演示了这些命令。为了节省篇幅，本附录的一些演示（包括这个）使用了 Python 的 `-q` 标志来抑制会话启动时的消息；这只是外观上的，可加可不加。`py` 命令严格来说是随 Python 一起安装的 Windows 启动器（Windows launcher）的一部分。默认情况下，启动器运行你 PC 上安装的最近版本的 Python；但如果你装了不止一个版本，也可以指定要运行的版本（例如 `py -3` 运行最新的 3.X，`py -3.8` 运行它的旧版）。如果你只有一个 Python，或者想用最新的，`py` 就足以启动交互式会话或脚本了。如果你在安装时勾选了把 Python 加入系统 PATH，也可以使用 `python` 命令启动 Python，不过那不过是多打几个字罢了（`python3` 也行，但只有按前面那个注意装的是商店版才行）。而且和 Unix 上一样，你可以轻松地把脚本的打印输出保存到文件里，只需在命令末尾加上 `> filename.txt`（关于这种流重定向（stream redirection）的更多内容见第 3 章）。

代码分析

```
py                rem Python >>>
py script.py      rem Python script.py
py -q            rem -quiet
py -3            rem Python 3.X
py -3.8          rem Python 3.8
python           rem py" PATH"
python3          rem Microsoft Store
py script.py > out.txt rem out.txt
```

逐条讲解：

1. `py` 是 Windows 特有的命令，来自随 Python 安装的 `launcher`（启动器）。它先查系统里装了哪些 Python 版本，然后默认挑最新的那个来启动。启动器对新手最大的意义是：不用管 `PATH` 配得对不对，敲 `py` 永远能找到 Python。
2. `py -3 / py -3.8`：参数 `-3` 把版本过滤到 3.X 系，`-3.8` 更精确到次版本。这是多版本共存时代的核心工具——可以想象成“遥控器上选频道”。
3. `> filename.txt`：这是操作系统 shell（不是 Python）提供的输出重定向（`redirection`）——把本来打印到屏幕的标准输出（`stdout`）转存到文件。第 3 章会展开讲标准流。
4. `python` 与 `python3` 的差别：`python` 需要 `PATH` 正确配置；`python3` 在 Windows 上根本不是惯例（是商店版的“赠品”）。这正是 Windows 与 Unix 用词差异的缩影。

深度理解

·核心概念：Windows 上最重要的命令是 `py` (Windows launcher)，而不是 Unix 世界习惯的 `python3`。它同时解决"找 Python"和"选版本"两个问题。

·底层实现：launcher 是一个独立的 exe (`py.exe`)，它扫描注册表与文件系统定位已安装的 Python，解析命令行中的 `-3.X` 参数，把控制权交给对应解释器。它甚至支持读取脚本首行的 `#!`（下一小节会讲）。

·设计思想：Unix 靠 PATH 和符号链接管理多版本，Windows 的 PATH 生态混乱，微软又不管——Python 官方于是造了个"启动器"来兜底。这是"平台缺陷催生官方工具"的典型例子。

·实际问题：为什么命令行"移动部件最少"？因为命令行的依赖只有"命令 + 文件名"两样；IDE 则需要理解工程、调试器、运行配置等一系列概念。

·初学者误区：①以为 `python3` 在 Windows 上通用 (Unix 习惯惯性思维)；②以为 `py` 与 `python` 是同一件事 (`py` 才是 launcher 的正牌入口)；③输错版本号就以为 Python 没装好。

A.2.5 文本编辑器与开始菜单 REPL (Text Editors and the Start-Menu REPL)

英文原文

However, if you opt to go the command-line route, you'll also need to choose a text editor to create files of code you wish to save (i.e., scripts to run and modules to import). As demoed in Figure A-2, Windows Notepad suffices, but any Windows text editor will fit the bill. To use Notepad, launch it from your Start menu (search there if needed) or by typing its name in a command line, with or without a filename to edit. Besides command lines, you can also start Python's interactive session by clicking the Python 3.12 (64-bit) (or similar) item in its Start-menu entry, shown in Figure A-3. This starts the usual Python REPL (Read-Eval-Print Loop) interactive session with its `>>>` prompt, just like an explicit `py` command in Command Prompt or PowerShell. You'll still use `py` commands or other techniques, though, to run code files. In all console REPLs on Windows, type or tap the two-key combo `Ctrl+Z` (followed by `Enter`) at the `>>>` prompt to exit a Python interactive session or simply close the hosting window.

中文翻译

不过，如果你选择走命令行这条路，你还需要选一个文本编辑器（text editor），用来创建你想要保存的代码文件（即要运行的脚本和要导入的模块）。正如图 A-2 演示的，Windows 的记事本（Notepad）就够用，任何 Windows 文

本编辑器都能胜任。使用记事本时，可以从开始菜单启动它（需要的话可以在那里搜索），也可以在命令行里输入它的名字，后面带或不带要编辑的文件名都行。除了命令行，你还可以通过点击开始菜单条目中的 Python 3.12 (64-bit)（或类似）项来启动 Python 的交互式会话，如图 A-3 所示。这会启动常规的 Python REPL（Read-Eval-Print Loop，读取-求值-打印循环）交互式会话，带有 `>>>` 提示符，与在命令提示符或 PowerShell 里显式输入 `py` 完全一样。不过，要运行代码文件，你仍然要用 `py` 命令或其他技术。在 Windows 上所有控制台 REPL 中，在 `>>>` 提示符下输入或敲击 `Ctrl+Z`（再按回车）即可退出 Python 交互式会话，或者干脆关掉承载它的窗口。

代码分析

```
notepad                                rem notepad script.py
rem "Python 3.12 (64-bit)" = py
rem REPLCtrl+Z EnterWindows — Unix Ctrl+D
```

逐条讲解：

1. 记事本（Notepad）：作者用它演示“任何编辑器都行”。对纯文本的 `.py` 文件，记事本完全够用——但要注意 Windows 记事本默认保存为带 BOM 的 UTF-8，个别老版本甚至会破坏编码，Python 3.12 时代这已不是大问题。
2. 开始菜单的 REPL 入口：Python 3.12 (64-bit) 图标本质上是“固定了参数 `py -3.12` 的快捷方式”——它启动的就是同一个 REPL，没有什么神秘的额外能力。作者特意强调：运行代码文件仍然要靠 `py` 命令，图标只给你交互

式会话。

3. Ctrl+Z 退出：这是 Windows 控制台向 REPL 发送"文件结束 (EOF)"信号的惯例，Unix 上则是 Ctrl+D。按 Ctrl+Z 再回车，Python 解释器读到 EOF 标志，正常结束会话。

深度理解

·核心概念：命令行 + 文本编辑器 = 最少工具集 (two tools, one workflow)。REPL 用于"试"，脚本文件用于"存"。

·底层视角：REPL (读取-求值-打印循环) 是解释器最原始的交互形态：>>> 提示符下每输一行，Python 就编译一行、执行一行、打印结果。它是 Python "探索式编程"的物理载体。

·设计思想：为什么"任何编辑器都行"？因为 Python 的源码就是纯文本，.py 文件不包含任何专有二进制格式——这是 Python 与 VB、Delphi 这类"可视化 IDE 绑定语言"的根本差异。

·实际问题：Windows 上没有 Unix 的 Ctrl+D 习惯，所以 Python 团队在 Windows 上用 Ctrl+Z 表示 EOF。很多从 Windows 转到 macOS 的新手在这里栽跟头。

·初学者误区：①以为必须装 VS Code 才能写 Python (其实记事本就行，编辑器只是工具)；②以为点开始菜单图标就能跑脚本 (它只开 REPL)；③试图用 Ctrl+C 退出 REPL (那是中断当前代码，不是退出会话)。

A.2.6 IDLE: Python 自帶 IDE (IDLE)

英文原文

If command lines make you break out in hives, you can also run Python from graphical IDEs like PyCharm or Python's own IDLE. Of these, IDLE is included with Python for Windows and provides a simple but sufficient IDE for running this book's examples. Its utility partly overlaps with command lines (it's ultimately just a place to type and run code), but it also includes a Python-friendly text editor for code files and simplifies some common coding chores. Notably, it's able to launch program files without command lines. As examples, Figures A-4 and A-5 capture IDLE's interactive "Shell" and editor windows, respectively, with default configurations. You can start IDLE from Python's entry in your Start menu on Windows (try a search for "idle" there to locate and open IDLE quickly). The command `py -m idlelib.idle` also starts IDLE, for reasons covered elsewhere in this book (tl;dr: this is like a module import, but runs instead of importing), and right-clicks on code files in File Explorer can open IDLE too, but require registry edits today. IDLE's own Help menu comes with ample usage info that we'll defer to here, but one tip is worth a callout: a menu `Run → Run Module` (or its equivalent F5 shortcut key) in any editor window like that in Figure A-5 lets you launch a scrip

t without typing a command line. This runs the code in that window after it's been saved to a file if needed and routes the code's printed output back to the Shell window. Its Run...Customized version also lets you provide command-line arguments (see `sys.argv` in Python's manuals for details). Useful tricks to be sure, but even if you don't use IDLE to run code this way, it has additional tools we'll skip here for space, and its code editor alone makes for a compelling alternative to Notepad if you have no other option in mind for scripts and modules.

中文翻译

如果命令行让你浑身起鸡皮疙瘩，你也可以从 PyCharm 或 Python 自带的 IDLE 这类图形化 IDE 运行 Python。其中，IDLE 随 Windows 版 Python 一起提供，为运行本书的示例提供了一个简单但够用的 IDE。它的功能与命令行有部分重叠（归根结底它只是一个输入并运行代码的地方），但它还包含一个对 Python 友好的代码文件编辑器，并简化了一些常见的编码杂务。值得注意的是，它能够不借助命令行就直接启动程序文件。作为示例，图 A-4 和图 A-5 分别抓拍了默认配置下 IDLE 的交互式 "Shell" 窗口和编辑器窗口。你可以从 Windows 的开始菜单中 Python 的条目里启动 IDLE（在那里搜一下 "idle" 就能快速找到并打开它）。命令 `py -m idlelib.idle` 也能启动 IDLE，原因本书别处会讲（tl;dr: 这有点像模块导入（import），但它是"运行"而不是"导入"），在文件资源管理器中右键点击代码文件也能打开 IDLE，不过目前需要修改注册表。IDLE 自己的 Help 菜

单带有充足的用法说明，我们就不展开了，但有一个技巧值得点名：在任何像图 A-5 那样的编辑器窗口里，菜单 Run → Run Module（或其等效的 F5 快捷键）可以让你不敲命令行就能启动脚本。它会（如果需要的话）先把该窗口的代码保存成文件再运行，并把代码的打印输出送回 Shell 窗口。它的 Run...Customized（自定义运行）版本还允许你提供命令行参数（细节见 Python 手册里的 sys.argv）。这些确实是实用的技巧，但即使你不这样用 IDLE 运行代码，它还有更多工具，这里限于篇幅就不展开了；而且仅凭它的代码编辑器，在你对脚本和模块没有任何其他想法的情况下，它也比记事本更有吸引力。

代码分析

```
py -m idlelib.idle      rem "" IDLE— IDLE
rem F5 Run -> Run Module
rem Run -> Run...Customized sys.argv
```

逐条讲解：

1. `py -m idlelib.idle`：-m 的含义是“以模块（module）方式运行”。idlelib.idle 是标准库里的一个模块名。执行它等价于 `import idlelib.idle` 之后再调用其中的运行入口——这正是“像模块导入，但运行而非导入”那句话的含义。任何可运行的模块（例如 `py -m pydoc`、`py -m pip`）都能这样启动。

2. Run → Run Module (F5)：先自动保存当前文件（必要时），再用 python 文件路径的方式运行，把 stdout 捕获并显示在 Shell 窗口。F5 是 IDLE 里最高频的快捷键。

3. Run...Customized: 相当于命令行里 `py script.py` 参数1 参数2, 参数会被填进 `sys.argv` 列表, 供脚本内部用 `sys.argv[1:]` 读取。

深度理解

·核心概念: IDLE = "Shell 窗口 (交互式) + 编辑器窗口 (写文件) + 菜单串联两者"。它刻意做小, 只覆盖"编辑→运行→看输出"这条最短链路。

·底层实现: IDLE 本身是一个用 `tkinter` (Tk GUI 工具包) 写成的 Python 程序——用 Python 写出来的 IDE。Shell 窗口与编辑器窗口之间通过进程通信协作 (运行代码时启动子进程), 所以你会看到多个 python 进程。

·设计思想: IDLE 存在的理由不是"强大", 而是"零依赖": 随 Python 一同分发, 无需安装任何东西。它对这本书的价值是"打开就能用", 把注意力留给语法本身。

·实际问题: 在文件资源管理器右键用 IDLE 打开需要改注册表 (作者如实说明了); 日常最顺手的启动路径就是开始菜单搜索 "idle"。

·初学者误区: ①把 IDLE 的 Shell 窗口当成编辑器 (Shell 里敲的东西不会存进文件); ②以为 F5 能运行"未保存"的代码 (它会先要求保存); ③以为 `-m` 是 IDLE 专属——它是 Python 的通用运行机制。

A.2.7 文件名关联与点击运行 (Filename Associations and Clicking)

英文原文

Beyond `py`, `python`, `python3`, `Start`, and `IDLE` (which already qualifies as a Windows embarrassment of riches!), a Python program file can also be launched on Windows by typing just its name in a command line (e.g., `hack.py`), and by locating and clicking its name or icon in Windows File Explorer. These both work thanks to the magic of filename associations in Windows, which Python sets up automatically during its install: any file whose name ends in `.py` is routed to the `py` launcher when named or clicked. Clicking, however, comes with a drawback: printed output of programs you launch this way is lost on program exit, because the program's run window is closed. To keep the window (and hence output) open, simply add a call to Python's `input()` at the bottom of your script to pause for a user Enter-key press. As in Figure A-5, this call can be conditional on the platform to pause selectively (some code may warrant standard-stream TTY tests too). The `input()` trick won't help if the program commits an error (alas, the error messages may perish with the window before the pause is ever reached!), so running by clicks is usually best used only for graphical programs and others that log their errors to files. Tip for GUIs: a `.py` opens a console for standard stream IO when clicked, but a `.pyw` does not.

中文翻译

除了 `py`、`python`、`python3`、开始菜单和 IDLE（这已经算是 Windows 上的“幸福的烦恼”了！），Python 程序文件在 Windows 上还可以通过只在命令行里输入它的名字（例如 `hack.py`）来启动，也可以在 Windows 文件资源管理器中定位并点击它的名字或图标。这两种方式之所以可行，都要归功于 Windows 的文件名关联（filename associations）魔法——Python 在安装时会自动设置：任何以 `.py` 结尾的文件，在被输入名字或被点击时，都会转交给 `py` 启动器处理。然而，点击运行有一个缺点：以这种方式启动的程序，其打印输出会在程序退出时丢失，因为程序的运行窗口被关闭了。要让窗口（以及输出）保持打开，只需在脚本底部加一个对 Python `input()` 的调用，让它暂停等待用户按回车键。如图 A-5 所示，这个调用可以按平台做条件判断，有选择地暂停（有些代码可能还值得做标准流 TTY 测试）。如果程序报错，`input()` 这个技巧就帮不上忙了（唉，错误消息可能会在程序到达暂停点之前就随窗口一起消失了！），所以点击运行通常只适合图形程序，以及其他把错误记入文件的程序。给 GUI 的提示：点击时，`.py` 会打开一个用于标准流 IO 的控制台，而 `.pyw` 不会。

代码分析

```
import sys

print('Hello, world!')

# Windows ""
if sys.platform == 'win32':
    input('Press Enter to exit...') #
```

说明：这是书中描述的模式（"这个调用可以按平台做条件判断"）的典型写法示意，供理解用途。

逐条讲解：

1. `sys.platform == 'win32'`： `sys.platform` 是 Python 启动时探测到的宿主平台标识。Windows 一律是 `'win32'`（哪怕你是 64 位系统，这名字是历史遗留）；Linux 是 `'linux'`，macOS 是 `'darwin'`。这是"按平台条件化"的标准手段。
2. `input('Press Enter to exit...')`： `input()` 会读取一行标准输入直到用户按回车。放在脚本末尾，窗口就会停在那里等着——输出自然"幸存"下来。
3. `.py` vs `.pyw`： 文件关联里 `.py` 绑定到带控制台的 `py.exe`，双击会先弹出一个黑色控制台窗口（用于标准流 IO）； `.pyw` 绑定到 `pythonw.exe`（无控制台版本），适合纯 GUI 程序，不弹黑框。

深度理解

- 核心概念： Windows 的"文件名关联"（双击即运行）是 Python 安装器自动注册的：`.py` → `py.exe`，`.pyw` → `pythonw.exe`。这是 GUI 时代"双击图标"习惯的延续。
- 底层实现： 文件名关联本质上是注册表里的映射：扩展名 → 可执行程序。所以作者说右键用 IDLE 打开要"改注册表"，而安装器替你写好了 `.py` 的关联。
- 设计思想： 作者对点击运行整体持保留态度——因为输出会随窗口关闭而蒸发，连报错都留不下。这是"图形化便利

"与"调试可观测性"之间的权衡。

·实际问题: `input()` 暂停技巧对"正常结束"的程序有效; 对崩溃(异常)的程序无效——异常堆栈打印完窗口就关了。所以作者建议: 点击运行只用于图形程序。

·初学者误区: ①双击 `.py` 文件发现"窗口一闪而过"就以为程序没写对——其实只是没加 `input()` 暂停; ②以为 `.pyw` 也能打印调试信息到控制台(它根本没有控制台); ③用点击运行代替命令行跑书里的示例(作者明确说本书示例最好用命令行跑)。

A.2.8 高级技巧: shebang、环境变量与 `pywin32` (Advanced Tips)

英文原文

Before we move on, here are some advanced but useful tips for Python on Windows: Script `#!` lines: The `py` Windows launcher treats the first line in a script's file as special if it begins with `#!`. This line can name the version and location of the Python to be used to run the file's code (e.g., `#!/python3.12`), and all the usual Unix-style lines work (e.g., `#!/usr/bin/python3.12`). This line is entirely optional and no different than naming a Python in the command line used to launch it with `py` (e.g., `py -3.12 hack.py`) but may be useful when there are multiple Pythons installed on your PC, especially when scripts are run with clicks instead of com

mand lines. Environment variables: Windows command-line interfaces use the PATH (a.k.a. Path) environment variable to locate named programs like python: every folder on this list is searched. This is normally set up for Python automatically during its install if you opt in, but you can tweak it later yourself in Settings (search for "environment variable" there). In the same way, you may also create or mod PYTHONPATH, used to locate imported modules (per Chapter 22), as well as PYTHONUTF8 and PYTHONIOENCODING, used on Windows to specify the default Unicode encoding of files and redirected streams (this convoluted story has changed in 3.X often and will again; see Chapter 37). Other Windows options: Most Python code works the same on Windows as on other platforms, especially if it uses Python's portable system tools in modules like os. In cases where Windows-specific tools are required, though, the pywin32 third-party extension allows your Python programs to access many Windows APIs directly. For more about using Python on Windows, try python.org's HOWTO as well as the copious resources on the web (with the usual caution about vetting their copious sources). Here, let's move on to the next PC platform on our tour.

中文翻译

在继续之前，这里有一些关于 Windows 版 Python 的高级但实用的技巧：脚本 `#!` 行：如果脚本文件里的第一行以 `#!`

开头，py Windows 启动器会把它当作特殊行处理。这一行可以指定用来运行文件代码的 Python 的版本和位置（例如 `#!/python3.12`），而且所有常见的 Unix 风格写法都有效（例如 `#!/usr/bin/python3.12`）。这一行完全是可选的，与在命令行里用 `py` 显式指定 Python（例如 `py -3.12 hack.py`）没有区别，但在你的 PC 上装有多多个 Python 时可能很有用，尤其是当脚本是靠点击而不是命令行运行时。

环境变量：Windows 命令行界面使用 `PATH`（也叫 `Path`）环境变量来定位像 `python` 这样的具名程序：这个列表里的每一个文件夹都会被搜索。通常安装时只要你勾选了，Python 就会被自动配置好；但之后你也可以在“设置”里自行调整（在那里搜“environment variable”即可）。同理，你还可以创建或修改 `PYTHONPATH`（用于定位要导入的模块，见第 22 章），以及 `PYTHONUTF8` 和 `PYTHONIOENCODING`（用于在 Windows 上指定文件和重定向流的默认 Unicode 编码——这段曲折的历史在 3.X 时代变过多次，以后还会变；见第 37 章）。其他 Windows 选项：大多数 Python 代码在 Windows 上与其他平台上的表现一致，尤其是那些使用 `os` 这类模块中可移植系统工具的代码。不过，在确实需要 Windows 专用工具的情况下，`pywin32` 第三方扩展能让你的 Python 程序直接访问许多 Windows API。关于在 Windows 上使用 Python 的更多信息，可以试试 `python.org` 的 HOWTO 以及网上大量资源（照例要谨慎甄别这些众多的来源）。好了，让我们继续巡游到下一个 PC 平台。

代码分析

```
#!/usr/bin/python3.12 # shebang Windows py Pyt...
#!/python3.12        # Windows launcher
```

```
rem py -3.12 hack.py — shebang
rem
rem PATH python/py
rem PYTHONPATH import
rem PYTHONUTF8 1 UTF-8
rem PYTHONIOENCODING utf-8
```

逐条讲解：

1. `#!/python3.12`: Windows launcher 认识两种 shebang: 简写 (`#!/python3.12`) 和完整 Unix 路径 (`#!/usr/bin/python3.12`——实际上那个路径在 Windows 上不存在, launcher 只是按规则提取 `python3.12` 这段名字)。这让"双击运行"也能选对版本。
2. `PATH`: shell 找可执行程序时, 按 `PATH` 列出的文件夹逐个搜索。勾选 "Add Python to PATH" 就是往这个列表里塞一个文件夹。找不到 python 报'`python`' is not recognized, 九成是 `PATH` 没配好。
3. `PYTHONPATH`: 和 `PATH` 不同, 它管的是 Python 自己的 `import` 语句——解释器启动时把它的条目加进模块搜索路径。第 22 章会专门讲。
4. `PYTHONUTF8` / `PYTHONIOENCODING`: Windows 历史遗留的编码混乱 (`cp936`、`cp1252`.....) 让文件读写和重定向流的编码选择变得曲折。这两个变量让你把默认编码钉死在 `UTF-8`, 是现代开发者的"止痛药"。

深度理解

- 核心概念：这三个高级技巧对应三个维度的"选择权"：版本选择（shebang）、查找路径（环境变量）、系统能力（pywin32）。
- 底层实现：launcher 会在脚本执行前扫描前两行，寻找 `#!` 前缀并解析版本名，然后在已安装版本里做精确匹配——这就是"双击也能多版本共存"的机制。环境变量则是所有进程启动时从父进程继承的全局键值表。
- 设计思想：这些机制全都"可选但存在"——Python 不强迫你用，但当你遇到多版本、编码乱码、Windows 专有功能时，它们是官方留下的逃生通道。
- 实际问题：编码问题是 Windows 老用户最深的痛：文本文件、控制台输出、重定向到文件的编码不一致就会乱码。PYTHONUTF8=1 是 3.7+ 之后的现代答案。
- 初学者误区：①以为 shebang 是"注释随便写"（对 launcher 而言它是配置，写错可能选错解释器）；②乱改 PATH 把系统的 python 指向了别的程序（Windows 上同名程序冲突很常见）；③以为 os 模块的代码在 Windows 上跑不了（恰恰相反，os 是跨平台的）。

A.3 在 macOS 上使用 Python (Using Python on macOS)

A.3.1 安装：预装迷局与 python.org 安装器 (Installation)

英文原文

As a Unix-based platform, macOS is well suited to Python, open source, and software development in general. At this writing, newer macOS PCs no longer come with a Python preinstalled (and if it seems they do, it's just a stub for installing unrelated toolsets, per the note ahead). Older macOS systems do have a Python, but it's the now-dated 2.X, and may issue a deprecation warning when launched (in other words, you can't use it to run this book's code, and it's not long for the macOS world). Hence, an install is required. As for Windows, the recommended way to install Python 3.X for macOS begins with a visit to the Downloads page at python.org. There, you'll fetch a self-installer for macOS that you'll run to install Python, along with its IDLE GUI and standard library (including the library's tkinter GUI toolkit). The Python you'll get today is a Universal 2 binary that runs natively on macOS PCs using both newer Apple Silicon (ARM) and older Intel (x86) chips, and can be run in the Rosetta 2 emulator (see the web for more on all such terms). Figure A-6 captures the install process on macOS. NOTE — Installation convolution: If you type `python3` in macOS's Terminal before running the python.org install, you ma

y get an Apple popup that asks if you want to install Python as part of Xcode's "command line developer tools." This is similar in spirit to the Microsoft Store redirect on Windows of the preceding section—and similarly confusing! On macOS, though, most users should ignore the offer and instead install Python from python.org as described here because it makes your Python independent of the version and configuration choices made by a tools package. This is also true if you already have Xcode and its Python: install a new Python from python.org for generally better control.

中文翻译

作为基于 Unix 的平台，macOS 非常适合 Python、开源以及一般意义上的软件开发。在本书写作时，较新的 macOS PC 不再预装 Python（就算看起来装了，那也只是一个用来安装无关工具集的“空壳”，详见下面的注意）。较老的 macOS 系统确实带 Python，但那是早已过时的 2.X，启动时可能还会弹出弃用警告（换句话说，你不能用它来运行本书的代码，它在 macOS 世界里也时日无多了）。因此，安装是必须的。和 Windows 一样，macOS 版 Python 3.X 的推荐安装方式，是从访问 python.org 的 Downloads 页面开始。在那里，你会下载一个 macOS 自安装程序，运行它即可安装 Python，同时装上它的 IDLE GUI 和标准库（包括库里的 tkinter GUI 工具包）。今天你拿到的 Python 是 Universal 2 二进制，既能原生运行在使用较新的 Apple Silicon (ARM) 芯片上的 macOS PC，也能跑在较老的 Intel (x86) 芯片上，还可在 Rosetta 2 模拟器中运行

(这些术语的更多信息请上网查)。图 A-6 抓拍了 macOS 上的安装过程。注意——安装的弯弯绕绕：如果在运行安装器之前，在 macOS 的终端 (Terminal) 里输入 python 3，你可能会收到一个 Apple 弹窗，问你是否要安装作为 Xcode 的"命令行开发者工具"一部分的 Python。这与上一节 Windows 上 Microsoft Store 的重定向在精神上如出一辙——同样令人困惑！不过，在 macOS 上，大多数用户应该无视这个提议，按这里说的方式从 python.org 安装 Python，因为这能让你的 Python 独立于工具包做出的版本和配置选择。即使你已经装了 Xcode 和它带的 Python，也应从 python.org 装一个新的 Python，以获得普遍更好的控制。

深度理解

·核心概念：macOS 有三种隐藏陷阱：新机没有 Python (只有"空壳")；老机的 python 是 2.7；Xcode 弹窗想塞给你一个"工具集附赠 Python"。结论：必须自己装，且首选 python.org。

·底层视角：所谓"空壳 (stub)"指的是 /usr/bin/python 3 只是一个占位可执行文件，运行它只会触发"安装命令行开发者工具"的引导弹窗。而 Universal 2 是苹果的"胖二进制"格式——一个文件里打包 ARM 和 x86 两套指令，系统自动挑合适的执行。

·设计思想：苹果把 Python 当作"系统组建的依赖"而非"开发者的第一公民"，所以版本被工具集 (Xcode) 绑架。python.org 安装的 Python 装在独立目录 (/Library/Frameworks) 里，版本和配置完全由你掌控。

·实际问题：老 Mac 用户敲 python 可能真的唤起 Python 2 并收到 deprecation 警告——这正是 2→3 时代最著名的过渡陷阱，作者特意提醒。

·初学者误区：①看到苹果弹窗就点“安装”（你会得到一个被 Xcode 约束的 Python）；②以为 python 和 python 3 没区别（老系统上它们可能真的是两个不同的 Python！）；③以为 Mac 上不需要装 Python（新系统真没有）。

代码分析

本小节书中没有给出可直接运行的命令——安装过程主要是图形化点击安装器。真正动手敲的命令（python3、python3 script.py）在下一小节 A.3.2 的代码分析中展开。

A.3.2 命令行运行与 Terminal (Running by Command Lines)

英文原文

After the install, you can start Python both with its entries in Launchpad, which mostly mirrors your PC's Applications folder in Finder, as well as command lines in Terminal, which provides a standard Unix command-line shell. Of these, Terminal may be the most basic way to get started with this book's examples on macOS. To run code by command line on macOS, open Terminal by clicking its entry in either Launchpad's Ot

her folder, or Applications → Utilities in Finder. Then, type `python3` to start a Python interactive session where you can type and run code, or `python3 script.py` to launch a code file. This works because `python3` is added to your system PATH by the install (and as a caution, `python` may mean Python 2 on older PCs). Figure A-7 demos Python commands live on macOS. As usual on Unix-based systems, type keys combo Control+D at the `>>>` prompt to exit a Python interactive session here (yes, this differs from Windows), and alias `python3` to something shorter in your shell's startup files if seven characters is too much (e.g., alias to `py`, if you want to avoid some disorientation when hopping to and from Windows).

中文翻译

安装完成后，你可以通过 Launchpad 里的条目启动 Python——Launchpad 基本映射了访达（Finder）里的应用程序（Applications）文件夹；也可以通过终端（Terminal）里的命令行——终端提供一个标准的 Unix 命令行 shell。其中，Terminal 可能是 macOS 上入门本书示例的最基本方式。要在 macOS 上用命令行运行代码，请打开终端：点击它在 Launchpad 的其他（Other）文件夹中的条目，或访达中的应用程序 → 实用工具（Applications → Utilities）。然后输入 `python3` 启动一个 Python 交互式会话，在其中输入并运行代码；或者输入 `python3 script.py` 启动一个代码文件。这样做之所以有效，是因为安装程序把 `python3` 加进了你的 PATH（而且提醒一句：在较老的系统上

，python 可能指的是 Python 2)。图 A-7 在 macOS 上现场演示了 Python 命令。与 Unix 系系统的惯例一致，在这里的 >>> 提示符下按 Control+D 组合键即可退出 Python 交互式会话（是的，这与 Windows 不同）；如果七个字符实在太多，可以在 shell 的启动文件里给 python3 设一个更短的别名（例如别名设成 py，这样你在 Windows 和 Mac 之间来回切换时还能少点迷失感）。

代码分析

```
python3          # Python >>> 3.12
python3 script.py #
# REPL Control+D Unix Windows Ctrl+Z
# python3 shell ~/.bash_profile ~/.zprofile
alias py=python3  # py python3
```

逐条讲解：

1. python3 是 macOS 的默认命令：与 Windows 的 py 不同，macOS 的惯例是 python3（因为 python 历史上指 3，3.9 时代必须用带 3 的名字区分）。python.org 安装器会把它装进 /usr/local/bin，而该目录默认在 PATH 中。
2. Control+D 退出：Unix 的 EOF 信号是 Ctrl+D——注意 macOS 键盘上是 Control 键，不是 Command 键。Windows 的 Ctrl+Z 习惯在这里无效。
3. alias py=python3：alias（别名）是 shell 的功能——py 只是 python3 的“外号”。把这一行写进 shell 启动文件（Bash 的 ~/.bashprofile 或 Zsh 的 ~/.zprofile），每次打开新终端都生效。这是“跨平台肌肉记忆”的官方

推荐解法。

深度理解

·核心概念：macOS 运行 Python 的最小路径 = 终端 (Terminal) + python3 命令。终端提供的是标准 Unix shell——与 Windows 的 cmd/PowerShell 是两种完全不同的命令行文化。

·底层实现：终端里的 shell (默认 Zsh) 按 PATH 查找 python3，找到后作为子进程运行，标准输入输出直接接在终端上——所以 print() 的输出出现在终端里。Ctrl+D 是在终端驱动层发送 EOF。

·设计思想：macOS 保留了 Unix 的"命令即工具"哲学：小而专的命令 (python3、vi、chmod、man) 通过 shell 组合成工作流。alias 机制则是"工具可以别名"哲学的体现。

·实际问题：从 Windows 过来的读者最大的错乱点：python vs python3 的命名习惯、Ctrl+D. vs Ctrl+Z 的退出键。作者用"少点迷失感"精准描述了这种跨平台水土不服。

·初学者误区：①按 Command+D 想退出 REPL (那是 macOS 的"删除光标后字符"快捷键，Control+D 才是 EOF)；②以为 python 一定可用；③alias 只写在当前终端里，关掉就没了 (要写进启动文件才持久)。

A.3.3 文本编辑器与 Python Launcher、IDLE (Text Editors, Launcher, and IDLE)

英文原文

When using command lines to run code, you'll also use a text editor to create files of Python code (scripts and modules) on macOS. Its built-in TextEdit suffices but isn't very code-friendly out of the box (e.g., you'll want to set a monospace font right away). Any macOS text editor is up to the task of editing Python code, including IDLE (up next), the vi and nano command-line-based editors familiar to Unix users, and other options you can explore on the web. After the install on macOS, you'll also find two tools for running Python code in other modes, available in both Launchpad and your Applications folder in Finder. As captured in both Figures A-8 and A-9, the Python Launcher allows you to run a file of Python code with either a click in Finder or a drag to its icon or name, and IDLE provides a basic edit-and-run IDE GUI for Python code. (Python itself, invoked by a `python3` command, shows up in `/Library/Frameworks`, though you don't normally need to care.) Figure A-10 shows IDLE on macOS. Launch it most easily with its Launchpad or Finder entries or by right-clicking (a.k.a. control-clicking) any Python code file in Finder and choosing IDLE in Open With (you can make this association permanent there if de

sired, and global in Finder's Get Info). Because IDLE is coded in Python as a tkinter GUI, it looks and works the same on all supported platforms (essentially, all PCs sans mobile platforms). As elsewhere, it allows you to edit and run files of Python code without having to use command lines or other editors. See IDLE's earlier Windows coverage for more tips; as noted there, IDLE can serve as an IDE-friendly code editor, even if you run files by command lines, clicks, or drags.

中文翻译

用命令行运行代码时，你还需要一个文本编辑器（text editor）来在 macOS 上创建 Python 代码文件（脚本和模块）。它自带的文本编辑（TextEdit）够用，但开箱即用的“代码友好度”不高（例如，你马上就会想设置等宽字体）。任何 macOS 文本编辑器都能胜任编辑 Python 代码，包括 IDLE（马上讲到）、Unix 用户熟悉的基于命令行的 vi 和 nano 编辑器，以及你可以在网上探索的其他选择。在 macOS 上安装完成后，你还会发现两个以其他模式运行 Python 代码的工具，在 Launchpad 和访达的应用程序文件夹里都能找到。正如图 A-8 和图 A-9 所展示的，Python Launcher 允许你用访达中的点击，或拖拽到它的图标或名字上这两种方式运行一个 Python 代码文件；而 IDLE 为 Python 代码提供了一个基本的编辑-运行式 IDE GUI。（Python 自身，即由 python3 命令调用的那个，位于 /Library/Frameworks，不过你通常不需要关心。）图 A-10 展示了 macOS 上的 IDLE。最轻松的启动方式是使用它的 Launchpad 或访达条目，或按住 Control 键点击访达中的任意 Python

on 代码文件并选择打开方式 (Open With) 里的 IDLE (如果需要, 你可以把这次的关联设为永久, 也可以再在访达的"显示简介 (Get Info) "中设为全局)。因为 IDLE 是用 Python 以 tkinter GUI 的形式编写的, 所以它在所有受支持的平台上看起来和用起来都一样 (基本上就是在除了移动端"壮举"之外的所有 PC 上)。和其他平台一样, 它允许你编辑和运行 Python 代码文件, 而无需使用命令行或其他编辑器。更多技巧见前面 Windows 部分对 IDLE 的介绍; 正如那里所说, 即使你通过命令行、点击或拖拽来运行文件, IDLE 也可以充当对 Python 友好的代码编辑器。

深度理解

- 核心概念: macOS 提供三件套: TextEdit (系统编辑器)、Python Launcher (点击/拖拽运行器)、IDLE (跨平台 IDE)。加上终端的 python3, 四选一即可开跑。
- 底层实现: Python Launcher 是一个极小的 GUI 应用——把选中的 .py 文件作为参数传给 Python 解释器执行。Python 本体安装在 /Library/Frameworks 下的 framework (框架) 结构里, 这是 macOS 特有的打包布局。
- 设计思想: IDLE 是 tkinter 程序——同一份 Python 源码跑遍三大桌面平台, 界面一致。这也是"用 Python 写 Python 的 IDE"的最好例证, 顺带证明了 tkinter 的跨平台能力。
- 实际问题: TextEdit 默认用比例字体、还会自动把引号换成"智能引号", 对代码不友好——作者的建议是马上设置等宽字体。macOS 的"打开方式"关联可以设为永久 (Always 或 Get Info), 比 Windows 的注册表编辑友好得多

·初学者误区：①在终端里敲 TextEdit 打不开文件（终端不认得这个命令；要用图形界面就直接点访达）；②想把右键（Control 键点击）当 Windows 的右键用——在访达里对代码文件 Control 键点击就能看到“打开方式”，这是打开 IDLE 的最快路径。

代码分析

```
open -e script.py          # TextEdit  script.py-e ...
open -a "Python Launcher" script.py  # Python Launcher
idle                        # IDLE
```

命令分析：把上述终端命令与 Windows 的 notepad script.py、py script.py 对照，可以看到两者都是“编辑器 + 解释器”的两层结构，区别只在平台偏好的命令不同：macOS 用 open 命令按应用打开文件，而 Windows 直接用程序名。

A.3.4 高级技巧：shebang、点击运行、环境变量与 Homebrew (Advanced Tips)

英文原文

To wrap up, here are a handful of advanced usage tips, with macOS spins that also apply to other Unix platforms like Linux and Android coming up next: Script `#!` lines: If the first line of a script begins with `#!`, it's treated as special on macOS, just as it is on Windows.

On macOS, this is a function of the shell (e.g., Bash or Zsh) that's running your command lines, not Python. For instance, a top-of-script line `#!/usr/bin/python3.12` tells the shell which Python should run the rest of the file's lines. This isn't different than naming your Python in a command line explicitly (e.g., `/usr/bin/python3.12 hack.py`), but a `#!` line allows a script to be run by just its name (e.g., `hack.py`) if it's also made executable (see `chmod`). Running by clicks: Python code files run with a click in Finder, but you may have to choose the Python Launcher in Open With and make it permanent with Always (or Get Info). Unlike on Windows, output is retained in the resulting console window on exit and errors. A `#!` first line isn't required but can be used. Environment variables: On macOS, you can also set or change environment variables like `PATH`—used to locate programs like `python3` named in command lines, and `PYTHONPATH`—used to locate imported modules—with code in your shell's startup files (e.g. `~/bashprofile` or `~/zprofile`). `PATH` updates automatically by python.org installs. For pointers on the shell code used to set these variables, see the example in Chapter 22, as well as the web and your PC's docs (e.g., `info bash` and the unfortunately coded `man bash`). Other macOS options: For completeness, it's worth noting that there are additional platform-specific tools for Python on macOS (e.g., `PyObjC`), and the third-party Homebrew package manager provides an

entirely different install scheme for Python on macOS, which works equally well but has extra setup steps that make it better suited to more advanced readers. For space, we'll skip further details here; see the HOWTO at python.org and your local web search engine for more Python options on macOS.

中文翻译

作为收尾，这里有几个高级使用技巧，带有 macOS 特色，同时也适用于接下来要讲的 Linux、Android 等其他 Unix 平台：脚本 `#!` 行：如果脚本的第一行以 `#!` 开头，它在 macOS 上会被特殊对待，和 Windows 上一样。但在 macOS 上，这是运行你命令行的 shell（例如 Bash 或 Zsh）的功能，不是 Python 的。例如，脚本顶部的一行 `#!/usr/bin/python3.12` 告诉 shell 应该用哪个 Python 来运行文件其余的行。这与在命令行里显式指定你的 Python（例如 `/usr/bin/python3.12 hack.py`）没有区别，但 `#!` 行允许脚本只靠它的名字（例如 `hack.py`）就能运行——前提是它还被赋予了可执行权限（见 `chmod`）。点击运行：Python 代码文件可以在访达中点击运行，但你可能需要在“打开方式”里选择 Python Launcher，并用始终（Always）（或“显示简介”）把它设为永久。与 Windows 不同，输出在退出和报错时都会保留在生成的控制台窗口里。`#!` 首行不是必须的，但可以用。环境变量：在 macOS 上，你也可以用 shell 启动文件（例如 `~/.bashprofile` 或 `~/.zprofile`）里的代码来设置或修改环境变量，比如 `PATH`——用于定位命令行里指定的 `python3` 这类程序；以及 `PYTHONPATH`——用于定位要导入的模块。`PATH` 由 python.org 安装程序

自动设置。至于设置这些变量的 shell 代码怎么写，参见第 22 章里的示例，以及网上和你电脑的文档（例如 `info bash` 和写得比较糟的 `man bash`）。其他 macOS 选项：为了完整性，值得一提的是 macOS 上还有一些平台专用的 Python 工具（例如 PyObjC），而第三方包管理器 Homebrew 为 macOS 上的 Python 提供了一套完全不同的安装方案，它同样好用，但有额外的安装步骤，更适合高级读者。限于篇幅就不展开更多细节了；更多关于 macOS 的 Python 选项请见 python.org 的 HOWTO 和你的本地网络搜索引擎。

代码分析

```
#!/usr/bin/python3.12  # shebang shell Python

/usr/bin/python3.12 hack.py  #
chmod +x hack.py            # ./hack.py
# ~/.bash_profile ~/.zprofile
export PATH="/usr/local/bin:$PATH"      # PATH
export PYTHONPATH="/path/to/my/modules" #
info bash                             # bash info
man bash                             # bash —""
```

逐条讲解：

1. shebang 在 Unix 上是内核/shell 的职责：与 Windows launcher 不同，Unix 上 `#!` 由内核在 `exec` 程序时解析——文件可执行时，内核读第一行，用其后的程序执行该文件。所以作者强调这是“shell/系统的功能，不是 Python 的”。

2. `chmod +x hack.py`：Unix 的可执行是一种权限位，不是“文件关联”（Windows 思路）。`+x` 给文件加上执行

权限后，`./hack.py` 才能运行；配合 `#!` 行，系统才知道用什么解释器。

3. `export PATH=...`: `export` 把变量从 shell 传给它的子进程——`python3` 的查找就是子进程用该 `PATH` 做的。`PYTHONPATH` 则是 Python 解释器启动时读取、并并入模块搜索路径的。

4. `man` vs `info`: Unix 的两套文档系统。作者说吐槽 `man bash` "编码糟糕"——这是老 Unix 手册公认的痛点，也意味着新手不必硬啃。

深度理解

·核心概念：macOS 高级技巧的核心差异是责权归属：Windows 上 launcher 管 shebang，Unix 上是内核/shell；Windows 用"文件关联"绑定程序，Unix 用"执行权限位"。

·底层实现：可执行脚本的完整链路是：`./hack.py` → 内核读 `#!` 行 → 用 `/usr/bin/python3.12` 作为解释器、把脚本路径作为参数 → Python 读取文件执行。三步缺一不可。

·设计思想：Unix 把"解释器选择"从文件系统层面解开，让任何语言（Python、Perl、Ruby）共用同一套 `#!` + `chmod` 机制——这是统一抽象的设计美学。

·实际问题：点击运行时 macOS 会保留输出窗口（比 Windows 友好）；Homebrew 是另一个安装入口——功能很好但引入依赖链，作者明确说"更适合高级读者"。

·初学者误区：①忘了 `chmod +x` 就 `./` 运行，报 "Permission denied" 还以为 Python 坏了；②把 Windows 的 python 习惯带到 Unix（在 Unix 里 python 可能真的是 Python 2）；③以为 man 是给新手看的（它更适合当参考书查）。

A.4 在 Linux 上使用 Python (Using Python on Linux)

A.4.1 安装与命令行运行 (Install and Run)

英文原文

Python is a staple on Linux, where it's used both for user applications and system tools. Because it's so ubiquitous on this Unix-based platform, Python may come preinstalled with your Linux distribution; type `python3` in a shell window (e.g., Terminal) to check. If you need to install manually, do the usual thing for your Linux flavor: a `sudo apt install python3` in Terminal does the deed in Ubuntu distributions (try `yum` on some others). Once you've got a Python 3.X installed, run code interactively and launch code files with the usual Terminal command lines, like those captured in Figure A-11. A `python3` (or a version-specific name) starts an interactive session for typing and running code, and adding a filename (e.g., `python3 script.py`) run

s a file. As on other platforms, your PATH setting is used by such commands to locate Python. As on all Unixes, the key combo Ctrl+D at >>> exits a Python REPL, and shorter shell aliases for python3 can avoid some typing.

中文翻译

Python 是 Linux 上的常客，既用于用户应用程序，也用于系统工具。由于它在这个 Unix 系平台上如此无处不在，Python 可能已经随你的 Linux 发行版预装好了；在 shell 窗口（例如终端）里输入 python3 即可检查。如果需要手动安装，就按你的 Linux 发行版的惯例来做：在 Ubuntu 系列发行版里，在终端执行 `sudo apt install python3` 即可搞定（其他一些发行版可以试试 `yum`）。一旦你装好了 Python 3.X，就可以用常规的终端命令行来交互式运行代码、启动代码文件，就像图 A-11 里演示的那样。python3（或版本专属的名字）启动一个用于输入和运行代码的交互式会话，加上文件名（例如 `python3 script.py`）则运行一个文件。和其他平台一样，这些命令靠你的 PATH 设置来定位 Python。和所有 Unix 一样，在 >>> 提示符下按 Ctrl+D 组合键退出 Python REPL，而给 python3 设置更短的 shell 别名可以少打几个字。

代码分析

```
python3                # + >>>
sudo apt install python3 # Ubuntu Debian/Ubuntu/Mint
yum install python3     # Fedora/RHEL/CentOS
python3 script.py       #
# REPLCtrl+D macOS Unix
```


逐条讲解：

1. `sudo apt install python3`: `sudo` (superuser do) 以管理员权限执行；`apt` 是 Debian 系的包管理器，`install` 子命令从软件仓库拉取并安装软件包。这是 Linux 世界"安装软件"的标准动作。
2. `yum`：另一个包管理器，属于 Fedora/RHEL 系。同一个 Linux 概念，不同的发行版用不同的包管理器——这是 Linux 生态"分裂又统一"的典型例子。
3. 检查是否预装：直接敲 `python3`——如果出来 `>>>` 提示符就说明已经装好；如果报 `"command not found"` 就需要安装。Linux 下 Python 是系统组件，很多系统工具脚本都依赖它。

深度理解

- 核心概念：Linux 下 Python 是家常便饭 (staple) ——既给用户跑程序，也给系统跑工具。因此大多数发行版要么预装、要么一条命令装好。
- 底层视角：`apt/yum` 装的是发行版维护的发行版打包版本——它和 `python.org` 的官方二进制可能存在小版本差异（例如 Ubuntu 可能打包 3.12.x 的某个补丁版），但语言层面完全兼容。
- 设计思想：Linux 的哲学是"软件归包管理器管"。你不需要去官网下载安装器，发行版替你做了版本筛选和安全补丁——这也是为什么作者只给了一条命令。
- 实际问题：Linux 没有 Windows 的 `py`、也没有 macOS

的安装器弹窗，一切回归最朴素的命令行；PATH、别名、Ctrl+D 这些机制和 macOS 完全一致——Unix 家族的血统在这里最纯粹。

·初学者误区：①以为 python 命令一定可用（很多发行版只提供 python3，python 可能是别名或不存在）；②sudo 失败就反复重试（需要输入密码，且可能没在 sudoers 组里）；③装完不知“装到哪了”（which python3 可以查看）。

A.4.2 文本编辑器与 IDLE (Text Editors and IDLE)

英文原文

When using command lines, you'll also need to use a text editor for scripts and modules. To make such a file of Python code, any Linux text editor will do, including the graphical Gedit default on Ubuntu and the shell-oriented vi and nano. You can also use Python's IDLE edit-and-run GUI on Linux. Launch it with an idle command line after installing it with `sudo apt install idle3`, or similar on other distributions. A code-file right-click and Open With in file explorers may start IDLE too. IDLE fine print: you may need to force versions with a more specific name (e.g., `idle-python3.12`); emojis might not work until a font install (e.g., `sudo apt install ttf-ancient-fonts-symbola`); and a platform-ag

nostic command line `python3 -m idlelib.idle` starts IDLE, too, per the Windows flavor noted earlier. Figure A-12 demos IDLE running on an Ubuntu Linux PC after all the kinks have been ironed out; it works the same on Linux as on Windows and macOS, and is covered in more detail in this appendix's Windows section.

中文翻译

用命令行时，你还需要一个文本编辑器来写脚本和模块。要创建这样的 Python 代码文件，任何 Linux 文本编辑器都行，包括 Ubuntu 默认的图形化 Gedit，以及面向 shell 的 vi 和 nano。你也可以在 Linux 上用 Python 的 IDLE 编辑-运行式 GUI。先运行 `sudo apt install idle3`（其他发行版类似）安装它，然后用 `idle` 命令行启动。在文件管理器里右键代码文件、选“打开方式”也可能启动 IDLE。IDLE 的“小字条款”：你可能需要用更具体的名字来强制指定版本（例如 `idle-python3.12`）；在安装字体之前 emoji 可能显示不出来（例如 `sudo apt install ttf-ancient-fonts-symbola`）；还有一个与平台无关的命令行 `python3 -m idlelib.idle` 也能启动 IDLE，正如前面 Windows 部分提到的那样。图 A-12 演示了把所有小问题都解决之后，IDLE 运行在 Ubuntu Linux PC 上的样子；它在 Linux 上与在 Windows 和 macOS 上表现完全一致，更详细的内容见本附录的 Windows 一节。

代码分析

```
sudo apt install idle3                # IDLE
idle                                  # IDLE
idle-python3.12                       # ID...
sudo apt install ttf-ancient-fonts-symbola # emoji
python3 -m idlelib.idle               # IDLE ...
sudo apt install python3-tk           # tkinter GUI ...
```

逐条讲解：

1. idle3 与 idle：发行版把 IDLE 打包成独立软件包，名字带 3 表示 Python 3 版。装好后 idle 命令即可启动 GUI。
2. idle-python3.12：当系统里装了多个 Python 版本时，直接敲 idle 可能挑错版本——用带版本号的命令精确指定。
3. ttf-ancient-fonts-symbola：emoji 字符属于特殊字体范围，Linux 默认字体未必覆盖；装一个 Symbola 字体即可显示——这是"GUI 也要依赖字体包"的 Linux 特色。
4. python3 -m idlelib.idle：与 Windows 的 py -m idlelib.idle 完全同一机制——"运行模块"（-m），不依赖任何启动器或快捷方式，在三大平台上都可用。
5. python3-tk：tkinter 在多数 Linux 发行版里是可选组件，需要单独安装——因为 Linux 把"图形"拆成了最小依赖。

深度理解

·核心概念：Linux 的 GUI 工具（IDLE、tkinter）不是"装 Python 就自带"的，而是独立的软件包，要用 apt 单独装

- 底层视角：Linux 发行版默认的 Python 是"最小化"的：解释器 + 标准库，GUI 部分（tkinter 需要 Tcl/Tk 运行时）被拆出去以减小系统体积。这体现了 Linux"按需组装"的包管理哲学。
- 设计思想：python3 -m 模块名这个机制在这里大放异彩——它让"启动 IDLE"变成"运行一个标准库模块"，从而在所有平台、所有安装方式下获得唯一一致的入口。
- 实际问题：Linux 的"版本错乱"问题突出：系统可能同时有 /usr/bin/python3（发行版版）和别处装的另一个 Python，所以作者提示"用更具体的名字强制版本"。
- 初学者误区：①装完 Python 发现 import tkinter 报错就以为 Python 坏了（其实只是缺 python3-tk 包）；②以为 idle 命令在所有发行版都存在；③把 emoji 显示问题当成 Python 的 bug（是字体问题）。

A.4.3 其他注意点：tkinter、shebang、点击运行与源码编译 (Other Notes)

英文原文

Also noteworthy on Linux: If you wish to use Python's portable tkinter GUI toolkit, you can install it separately if needed with `sudo apt install python3-tk` (or similar, in the richly bifurcated world of Linux package installs). As on macOS, a line starting with `#!` at the top

of your script can denote which Python runs the file, and environment variables like `PATH` and `PYTHONPATH` can be set in shell startup files, but the former is not usually required. See the macOS section's coverage of both topics and Chapter 22's `PYTHONPATH` example. Python files can be run by clicks on Linux too, but details vary. In Ubuntu's Files, "Run as a Program" runs a clicked Python file that has both executable permission (e.g., `chmod +x script.py`), and a `#!/...` first line that gives the path to Python, but the Windows caution about output disappearing on exit or error applies. It's not uncommon on Linux to build Python from its source code distribution, available at either python.org or GitHub. This entails a few simple command lines (`configure` and `make`) but is beyond the scope of both this chapter and most Python beginners; see the Downloads page at python.org for code and details. For more info about Python on Linux, try the web at large or python.org's HOWTO. Here, it's time to move ahead to Python's story on mobile.

中文翻译

Linux 上还有几点值得注意：如果你想用 Python 的可移植 `tkinter` GUI 工具包，需要的话可以单独安装它：`sudo apt install python3-tk`（或者类似命令——在 Linux 软件包安装那丰富多彩的分叉世界里）。这一节内容详见上文 A.4.2 的代码分析。和 macOS 一样，脚本顶部以 `#!` 开头的行可以指定用哪个 Python 运行这个文件；`PATH` 和 `PYTHON`

PATH 这类环境变量也可以在 shell 启动文件里设置——不过前者（#! 行）通常不是必需的。这两个主题的详细内容见 macOS 一节，PYTHONPATH 的示例见第 22 章。在 Linux 上，Python 文件也可以点击运行，但细节因环境而异。在 Ubuntu 的"文件"里，"作为程序运行（Run as a Program）"可以运行被点击的 Python 文件，前提是它既有可执行权限（例如 `chmod +x script.py`），又有给出 Python 路径的 `#!/...首行`——但 Windows 那节关于"输出在退出或报错时消失"的警告在这里同样适用。在 Linux 上从源码构建 Python 也并不少见——源码发布版可以在 python.org 或 GitHub 找到。这需要几条简单的命令（`configure` 和 `make`），但超出了本章和大多数 Python 初学者的范围；代码和细节请看 python.org 的 Downloads 页面。关于 Linux 上 Python 的更多信息，可以试试网上各处或 python.org 的 HOWTO。好了，是时候继续前进，去看看 Python 在移动端的故事了。

代码分析

```
chmod +x script.py      #
# #!/usr/bin/python3.12 #!/usr/bin/env python3"Run as ...
./script.py             #
# Python
./configure              # Makefile
make                     # Makefile configure + make
```

逐条讲解：

1. `chmod +x script.py`：Linux 的"可执行"是权限位；`.py` 文件加了 `+x` 并带 `#!` 首行后，就成了"可执行脚本"——与 Windows 的"文件关联"思路完全不同。

2. `./script.py`: `./` 表示"当前目录下的文件"——因为 Unix 的 PATH 通常不包含当前目录（安全考虑），直接敲 `script.py` 会找不到。

3. `./configure && make`: 源码构建的两板斧——`configure` 检测编译器和系统库、生成构建配置；`make` 调用编译链产出可执行文件。作者明确说：这对初学者超纲了，知道存在即可。

深度理解

·核心概念：Linux 把"安装 Python"的最终形态留给你选：包管理器装发行版（默认）、官网二进制（可选）、源码编译（进阶）。作者只推荐第一种。

·底层视角：`configure + make` 是 Unix 世界四十年的标准构建流程：`configure` 做"环境探测"，`make` 做"编译与组装"。今天很多 Python 包在源码里仍然保留这个流程，但普通用户永远不需要碰。

·设计思想：`tkinter` 单独安装、`#!/` 行可选、点击运行要"权限 + shebang"双条件——这些"麻烦"背后是 Linux 的最小权限原则与显式配置哲学。

·实际问题：点击运行时输出丢失的警告在 Linux 同样适用（和 Windows 一样，窗口关闭时输出蒸发）——作者特意把这条经验跨平台传递。

·初学者误区：①看到 `configure && make` 就以为自己必须学会编译 Python（完全不必）；②以为 `#!/usr/bin/env python3` 和 `#!/usr/bin/python3` 没区别（前者是"去 PATH 里找"，更通用）；③点击运行失败就怪 Python

——先检查 `chmod +x` 和 `#!` 行。

A.5 在 Android 上使用 Python (Using Python on Android)

A.5.1 总览: Python 应用三剑客 (Overview)

英文原文

Android is a secure derivative of Linux adapted for the unique constraints of mobile devices, and it is the most widely used operating system in the world at this writing. Despite this platform's Java and Kotlin programming-language biases, Python can be used as a first-class programming citizen on Android devices in both learning and development roles. This book's examples, for instance, will work well on your Android phone or tablet. To run Python locally on your Android, you'll first install an app that supports it from an app store like Play or F-Droid. Among these apps, Termux, Pydroid 3, and QPython all allow you to run Python code on Android directly in multiple modes. While we can't do justice to these and other Python apps, here's a quick rundown of two to get you started.

中文翻译

Android 是 Linux 的一个面向移动设备特殊约束而改良的安全衍生版，也是本书写作时世界上使用最广泛的操作系统。尽管这个平台在编程语言上偏爱 Java 和 Kotlin，Python 仍然可以在 Android 设备上作为"一等公民"参与编程，既用于学习也用于开发。例如，本书的示例在你的 Android 手机或平板上会运行得很好。要在你的 Android 上本地运行 Python，首先需要从 Play 或 F-Droid 这样的应用商店安装一个支持它的应用。在这些应用中，Termux、Pydroid 3 和 QPython 都允许你在 Android 上直接以多种模式运行 Python 代码。我们没法把这些应用一一讲透，这里快速介绍两个，帮你起步。

深度理解

- 核心概念：Android 底层就是 Linux 内核，所以"Unix 上能跑 Python"这件事在手机上有天然基础——只是被移动生态（应用商店、沙箱、触屏）包了一层壳。
- 底层视角：所谓"Python 应用"，本质上是把 Python 解释器（以及需要的标准库/扩展库）打进一个 Android APK 里。Termux 甚至带完整的包管理器，能像 Linux 一样 apt/pkg 装软件。
- 设计思想：Android 官方偏爱 Java/Kotlin，但 Python 通过"应用封装"路线成为一等公民——这正是"解释器可嵌入"设计的红利。
- 实际问题：应用商店的选择本身就是坑：Termux 的 Play 版已废弃，必须从 F-Droid 装；Pydroid 3 是"免费+广告

"的 freemium 模式。

·初学者误区：以为"手机上只能学不能练"——作者明确说本书示例在手机上就能跑；但也要理解手机不是主力开发环境。

A.5.2 Termux: 口袋里的 Linux (Termux)

英文原文

The free and open source Termux app for Android provides a full-featured Linux shell, toolset, and package manager. To use Python in Termux, first install the Termux app from the F-Droid store (its Play version is defunct). Then, open the app from your Apps screen, and install Python 3.X inside it with a `pkg install python` command line in its shell. Termux opens with a standard Bash command-line shell, where you can tap out commands to launch an interactive Python session with `python`, and run a file of code by adding a filename (e.g., `python script.py`). Stream redirection works as on all Unix, and command `python3` is the same as `python` if you prefer Unix uniformity and don't mind the extra tap. Both commands are automatically usable post install without PATH mods. You can use code files (scripts and modules) located in any folder Termux has access to (which generally means shared or app-private storage, per ahead) and make and change them either with separate text editor apps or within T

ermux itself using Linux text editors like vi and nano (install them in Termux with `pkg install` as needed). Termux also supports Android's Storage Access Framework to make its app-private storage visible to some file-explorer apps, although shared storage is more accessible and usable. Figure A-13 demos the Termux app running Python code and file on Android. As on all Unixes, the keys combo `Ctrl+D` at `>>>` ends a Python interactive session in Termux (via Termux's CTRL button or keyboards ahead), as does killing the app, and shell aliases can shorten the `python` (or `python3`) command. With apps, you'll generally use the version of Python provided; in Termux, this means one of the versions in package repos, but you may be able to build a newer one from source code. You can also install a host of extensions to use in your code by command line, with both Termux's `pkg` and Python's `pip`. Termux may be the path of least resistance for getting started with Python on Android. It has more features we'll largely skip here, including home-screen widgets that run Python scripts on taps, and all Linux concepts covered earlier apply, including `PYTHONPATH` and `PATH` environment-variable settings. Its chief downside for some users may be that it is limited to command lines sans its optional X Window System support, which is considerably complex to use; IDLE, for example, would be difficult at best to run in Termux.

中文翻译

Android 上的免费开源应用 Termux 提供了一个功能完整的 Linux shell、工具集和包管理器。要在 Termux 里用 Python，先从 F-Droid 商店安装 Termux 应用（它的 Play 版已经停更废弃）。然后从应用列表打开它，在它的 shell 里用 `pkg install python` 命令行安装 Python 3.X。Termux 打开后是标准的 Bash 命令行 shell，你可以敲出命令：用 `python` 启动 Python 交互式会话，加文件名（例如 `python script.py`）运行代码文件。流重定向（stream redirection）和所有 Unix 上一样工作；如果你偏爱 Unix 式的统一命名、也不在乎多敲几下，`python3` 命令和 `python` 是一样的。两条命令装完后开箱即用，不需要改 PATH。你可以使用 Termux 能访问的任何文件夹里的代码文件（脚本和模块）——这通常指共享存储或应用私有存储（后面会讲），可以用单独的文本编辑器应用创建和修改它们，也可以在 Termux 内部用 `vi`、`nano` 这类 Linux 文本编辑器（需要时用 `pkg install` 安装它们）。Termux 还支持 Android 的存储访问框架（Storage Access Framework），让它的应用私有存储对某些文件管理器应用可见，不过共享存储更容易访问和使用。图 A-13 演示了 Termux 应用在 Android 上运行 Python 代码和文件。和所有 Unix 一样，在 Termux 里于 `>>>` 提示符下按 `Ctrl+D` 组合键（通过 Termux 的 CTRL 按钮或前置键盘）即可结束 Python 交互式会话——杀掉应用也行——`shell` 别名可以缩短 `python`（或 `python3`）命令。用应用时，你一般只能使用它提供的 Python 版本；在 Termux 里，这意味着软件仓库中的某个版本，不过你也可能从源码构建更新的版本。你还可以用命

命令行安装大量扩展来供代码使用，Termux 的 pkg 和 Python 的 pip 都行。Termux 可能是 Android 上开始用 Python 阻力最小的路径。它还有更多特性我们基本略过了，包括点击即可运行 Python 脚本的主屏小部件（widget）；前面讲过的所有 Linux 概念同样适用，包括 PYTHONPATH 和 PATH 环境变量设置。对某些用户来说，它的主要缺点是基本限于命令行（除非用它的可选 X Window System 支持，但那用起来相当复杂）；例如 IDLE 在 Termux 里最多也只能勉强运行。

代码分析

```
pkg install python          # Termux Python 3.Xpkg Termu...
python                      # Python >>>
python script.py           #
python3                     # python Unix
pkg install nano            # nano vi/nano pkg
pip install requests        # Python pip
# REPLCtrl+DTermux CTRL
```

逐条讲解：

1. pkg install: Termux 自己的包管理器，仓库里预编译了 Python、vim、gcc 等大量软件——它让 Android 手机变成“口袋里的 Linux 开发机”。
2. python vs python3: 在 Termux 里两者等价（Termux 只提供 3.X），作者给的理由很实在：看你要不要“Unix 统一感”。
3. pip install: Python 生态的官方包安装器，pkg 管系统级软件，pip 管 Python 包——两层包管理并行。
4. 触屏的 Ctrl+D: Termux 的键盘工具条上有 CTRL 键

，按下后再点 D 就是完整的 Ctrl+D 组合——手机上的"键盘"是虚拟的，但语义与 PC 完全一致。

深度理解

·核心概念：Termux = 把"最小化 Linux 发行版"装进 App。它尊重 Unix 的一切规则（Bash、PATH、别名、流重定向），是移动端最接近"真开发环境"的选择。

·底层实现：Termux 不 root 手机——它在自己的应用私有目录里构建了一个完整的 Linux 用户态（bash、coreutils、编译器、包仓库），通过 Android 提供的终端界面与之交互。

·设计思想：作者称之为"阻力最小的路径"（path of least resistance）：无广告、免费、开放存储、包管理器齐全。它是移动端"最像 PC"的体验。

·实际问题：短板是 GUI——没有 IDLE、没有图形 IDE；X Window System 支持复杂到作者直接劝退。手机终究不是主力开发机。

·初学者误区：①从 Play 商店装 Termux（已废弃，要从 F-Droid 装）；②以为手机上的 Python 和 PC 的"不一样"（语法完全一样，只是环境变了）；③忽略主屏小部件等特性——那些是 Termux 的"彩蛋"。

A.5.3 Pydroid 3: 带 GUI 的 Python 应用 (Pydroid 3)

英文原文

If you're looking for something a bit more graphical, the Pydroid 3 app also provides a command-line shell and interactive Python session, but it adds a GUI IDE for editing and launching Python code. The IDE's edit/run window is captured in Figure A-14. Pydroid 3 is today installed from Play. Its shell and interactive session are less user-friendly than the richer command-line support in Termux, but its IDE may seem more comfortable for users unaccustomed to command lines. Similar in spirit to IDLE on PCs, this app's IDE allows you to edit Python code, and launch it with a simple (and yellow) button press. On top of its IDE, Pydroid 3 adds support for many popular tools, including scientific-programming libraries and, remarkably, Python's tkinter GUI toolkit. As in Termux, Python's version is preset in Pydroid 3 (though source builds are elusive), and Python's pip is available to install extensions (though with a dedicated GUI in this app). Fair warning: as a substantial trade-off, Pydroid 3 is also a free-mium app, which will flash rude full-page ads at you unless and until you pay a required fee—an unfortunately common paradigm in Android, which you'll have to weigh for yourself. In addition, Pydroid 3 has a history of waffling on support for storage access in response to Android and Play edicts. By contrast, Termux today is entirely free and void of ads, supports broad

d storage access, and chooses alternative app stores rather than limiting functionality for Android changes mandated by Play.

中文翻译

如果你想要更图形化一点的东西，Pydroid 3 应用也提供命令行 shell 和交互式 Python 会话，但它额外加了一个用于编辑和启动 Python 代码的 GUI IDE。IDE 的编辑/运行窗口见图 A-14。Pydroid 3 目前从 Play 商店安装。它的 shell 和交互式会话不如 Termux 那套丰富的命令行支持友好，但对不习惯命令行的用户来说，它的 IDE 可能感觉更舒适。与 PC 上的 IDLE 精神相似，这个应用的 IDE 允许你编辑 Python 代码，并按一个简单（而且是黄色的）按钮来启动它。除了 IDE，Pydroid 3 还支持许多流行工具，包括科学计算库，而且令人瞩目的是支持 Python 的 tkinter GUI 工具包。和在 Termux 里一样，Pydroid 3 里 Python 的版本是预置的（想从源码构建则很困难），Python 的 pip 可用于安装扩展（不过这个应用里有专用的 GUI 界面）。老实警告：作为一项重大的权衡，Pydroid 3 也是 freemium（免费增值）应用——除非并直到你支付所需的费用，否则它会向你弹出无礼的全页广告——这是 Android 上一个不幸的常见范式，需要你自己权衡。此外，Pydroid 3 在存储访问支持上有过摇摆不定的历史，因为要响应 Android 和 Play 的法规。相比之下，Termux 目前完全免费、没有广告、支持广泛的存储访问，并且宁愿选择替代应用商店，也不愿因 Play 强制要求的 Android 变更而限制功能。

深度理解

·核心概念：Pydroid 3 = "手机上的 IDLE"：命令行 + GUI IDE + 科学计算库 + tkinter。适合不习惯命令行的初学者。

·底层视角：它在 APK 里打包了一个完整的 Python 运行时和常用科学库（NumPy 等），IDE 按钮背后就是"保存文件 → 调用解释器运行 → 捕获输出"这一条 PC 上熟悉的链路。

·设计思想：作者的评价非常客观——功能互补（tkinter 是亮点），但商业模式是硬伤（强制广告）。这也是 Android 生态"免费午餐"的代价。

·实际问题：存储访问摇摆不定是个真实的坑——Android 各版本对存储权限的要求频繁变化，App 的适配常常滞后，可能导致你的代码文件"找不到"。

·初学者误区：①被全页广告吓到就卸载（其实付费解锁即可）；②以为 Termux 没有 GUI 就"差"——两者定位不同；③把"科学计算库支持"当成 Python 语言本身的能力（那是 App 打包了第三方库）。

A.5.4 其他工具与 CPython 官方支持 (Other Apps and CPython Support)

英文原文

See the web and app stores for info about other Pyth

on programming apps on Android omitted here for space. While you're at a store, you may also want to explore text editor apps like QuickEdit—which is able to colorize and run Python code; and alternative onscreen keyboards like Hacker's Keyboard—which adds PC keys not available in stock options but commonly used for coding (e.g., arrows and Ctrl). Some Python apps also include tools to augment onscreen keyboards that can be tailored or disabled, and Bluetooth keyboards and casting to larger screens can naturally aid usability too. It's also worth noting in closing that CPython plans to add Android to its list of officially supported platforms soon, which may foster additional options going forward. Moreover, although most beginners will use an app to run Python code on Android as described, it's also possible to build standalone apps for Android that are coded in Python but used like any other app. We'll return to this option at the end of this appendix after one last platform.

中文翻译

关于 Android 上其他 Python 编程应用的信息（这里限于篇幅略过），请看网上和应用商店。既然在逛商店，你还可以看看 QuickEdit 这类文本编辑器应用——它能给 Python 代码着色并运行；以及 Hacker's Keyboard 这类替代屏幕键盘——它补充了原装键盘没有、但编码常用的 PC 键（比如方向键和 Ctrl）。有些 Python 应用还内置了可定制或可关闭的屏幕键盘增强工具；蓝牙键盘和投屏到大屏幕也能

自然提升可用性。最后值得一提的是，CPython 计划很快把 Android 加入其官方支持平台列表，这可能为未来孕育更多选项。此外，虽然大多数初学者会像上面描述的那样用应用在 Android 上运行 Python 代码，但你也可以构建完全用 Python 编写、却像普通应用一样使用的 Android 独立应用。我们将在本附录的最后，在讲完最后一个平台之后回到这个选项。

深度理解

- 核心概念：移动端的"输入体验"是隐藏的生产力瓶颈：方向键、Ctrl、Tab 这些 PC 编码刚需在手机键盘上统统没有——所以有了 QuickEdit（编辑器）、Hacker's Keyboard（键盘）这样的"外设"应用。
- 底层视角：CPython 官方支持 Android 意味着：未来 Android 可能直接分发官方 Python 安装包（如同今天的 Windows/macOS 安装器），而不是只能靠第三方应用打包。
- 设计思想：作者反复埋下"独立应用（standalone apps）"的伏笔——这是移动端 Python 的"终极形态"：把 Python 程序打包成普通 App 分发。
- 实际问题：手机编码的物理短板（小屏幕、虚拟键盘）要靠外设补——蓝牙键盘 + 投屏是低成本方案。
- 初学者误区：以为手机上必须用"手机专用"的 Python 语法——完全没有这回事，代码与 PC 完全通用。

A.5.5 专栏: Android 的专有世界 (Android's Proprietary World)

英文原文

Python programmers should also be aware that Android imposes numerous constraints on apps, some of which may seem onerous to developers with backgrounds in more interoperable platforms. Most of these constraints are rationalized on the grounds of security or performance, but all reduce utility. Android's storage, for instance, is split into a shared and persistent area with controlled access, along with areas partly or wholly private to apps that may vaporize on app uninstalls. Hence, while POSIX file tools and paths do work on Android, Python code must take care to either use accessible folders or run proprietary Java API tools that request or use enhanced permissions. In addition, background or long-running processes may run a foul of limits; opinionated choices of tools and languages are nearly imposed on developers; and throttling for power, heat, memory, or other bias is a norm on most phones. On the upside, Android users can install apps outside its owner's store and can access the filesystem with numerous file-explorer apps. That makes Android more open than iOS today, but this is a large and fluid topic. If you care about using Python on mobiles, be sure to watch other resources for news o

n this front.

中文翻译

Python 程序员还应该意识到，Android 对应用施加了诸多限制，其中一些对来自更可互操作平台的开发者来说可能显得繁重。这些限制大多以安全或性能为由合理化，但全都削减了实用性。例如，Android 的存储被分割为访问受控的共享持久区，以及部分或完全对应用私有的区域——后者在应用卸载时可能烟消云散。因此，虽然 POSIX 文件工具和路径在 Android 上确实可用，Python 代码必须小心：要么使用可访问的文件夹，要么调用申请或使用增强权限的专有 Java API 工具。此外，后台或长时间运行的进程可能触犯限制；对工具和语言的专断选择几乎是强加给开发者的；为节电、散热、内存等原因的节流（throttling），在大多数手机上是一种常态。好的一面是，Android 用户可以在其拥有者的商店之外安装应用，也可以用众多文件管理器应用访问文件系统。这让 Android 今天比 iOS 更开放，但这是一个庞大而流动的话题。如果你关心在移动设备上使用 Python，务必关注其他资源在这个方向上的新闻。

深度理解

- 核心概念：Android 的"开放"是相对的——存储分区、后台限制、节流、语言偏好，都是压在开发者身上的"隐形税"。作者称之为"专有的世界（proprietary world）"。
- 底层实现：Android 的存储分三六九等：共享存储（媒体文件，权限受控）、应用私有存储（app-private，卸载即失）、以及"增强权限"通道（如 MANAGEEXTERNAL

LSTORAGE) 。POSIX 路径语法还在，但"能访问哪"由系统说了算。

·设计思想：安全与便利的权衡在这里走向极端——系统为了防恶意应用，把"自由"切成小块。Python 的 `open()` 照样工作，但"打开哪个目录"成了政治问题。

·实际问题：后台进程被系统随时杀死、CPU 被节流、你偏爱的工具语言可能不是平台第一选择——这些对"长任务脚本"尤其致命。

·初学者误区：①以为在 Android 上 Python 的 `os.listdir('/')` 能看到整个手机（只能看到权限允许的部分）；②把"应用卸载后文件消失"当成 bug（那是设计如此）；③以为 iOS 也一样开放（下一节见分晓）。

A.6 在 iOS 上使用 Python (Using Python on iOS)

英文原文

iOS—which includes its iPadOS offshoot in this guide—is a macOS derivative targeted at mobile devices. Like Android, it has limiting biases for programming languages (Swift and Objective-C), and it is even more strict about carving up storage into restricted app sandboxes with proprietary access rules and tools. Despite these constraints, though, your iPhone or iPad can be used to run Python code, too, including the code in this book. Like Android, you'll normally use Pyt

hon on your iOS devices by installing an app that runs Python code. Among these, Pythonista 3 provides an interactive Python session and a GUI for editing and running files of code, as in other IDEs. In addition, this app comes with access to native iOS features and a toolkit for building GUIs in Python for iOS. It also can share code files with the Files app to be opened with taps. To vet for yourself, fetch Pythonista 3 from the App Store. Figure A-15 shows this app in action (yes, on a humble and historical iPod). Be sure to also explore the other iOS options on the store; the Pyto app, for example, provides similar functionality, and comes with the Toga UI library for coding portable GUIs (there's more on Toga at standalone apps ahead). Apart from app choices and platform restrictions, using Python on an iOS device is largely the same as on Android and PCs, so we'll skip further details here. For more on using Python for iPhone and iPad, see the Apple App Store and the web at large. Like Android, iOS is also scheduled to be granted officially supported status in CPython soon, which may yield options impossible to predict today; watch the web for new developments. Also like Android, it's possible to package your Python programs as standalone apps for iOS, but we must move on to this appendix's next section to see how.

中文翻译

iOS——本指南中包括它的 iPadOS 分支——是面向移动设备的 macOS 衍生系统。和 Android 一样，它在编程语言上有明显的偏好（Swift 和 Objective-C），而且在把存储切分成受限的应用沙箱（app sandbox）这件事上更加严格，带有专有的访问规则和工具。尽管如此，你的 iPhone 或 iPad 同样可以用来运行 Python 代码，包括本书里的代码。和 Android 一样，在 iOS 设备上使用 Python 通常也是安装一个能运行 Python 代码的应用。其中，Pythonista 3 提供了一个交互式 Python 会话，以及一个像其他 IDE 那样用于编辑和运行代码文件的 GUI。此外，这个应用还能访问原生 iOS 特性，并带有一个用 Python 为 iOS 构建 GUI 的工具包。它还可以与文件（Files）应用共享代码文件，轻点即可打开。想亲自验证的话，请从 App Store 获取 Pythonista 3。图 A-15 展示了这个应用的运行画面（是的，是在一台朴素的、有历史感的 iPod 上）。也务必探索商店里的其他 iOS 选项；例如 Pyto 应用提供了类似功能，并且自带用于编写可移植 GUI 的 Toga UI 库（关于 Toga 的更多内容见后面的独立应用部分）。除了应用选择和平台限制之外，在 iOS 设备上使用 Python 与在 Android 和 PC 上大体相同，这里就不再展开更多细节了。关于在 iPhone 和 iPad 上使用 Python 的更多信息，请看 Apple App Store 和网上各处。和 Android 一样，iOS 也计划很快在 CPython 中获得官方支持地位，这可能催生今天无法预料的选项；请关注网上的新进展。和 Android 一样，你也可以把 Python 程序打包成 iOS 的独立应用，不过具体怎么做，要看本附录的下一节。

深度理解

·核心概念：iOS 是"更严格的 Android"：同样的移动端限制，但存储沙箱（sandbox）更硬、访问规则更专有。Python 的出路同样是应用封装：Pythonista 3、Pyto。

·底层视角：iOS 的沙箱把每个应用的存储与世界隔离——应用只能读写自己的私有目录和用户显式授予的文件。Python 的 open() 依然工作，但"能碰哪些文件"由 iOS 说了算。这也是后面"独立应用在 iOS 上很多跨平台代码跑不了"的根源。

·设计思想：苹果对"系统完整性"的执念导致任何"在设备上跑代码"的需求都得借道应用——Python 解释器被整个打进 App。Pythonista 3 的卖点正是"原生 iOS 能力 + Python"。

·实际问题：作者用"在 iPod 上演示"这个细节传递一个信号：这类应用是给小屏设备设计的，能跑但别指望生产力；iPad 体验远好于 iPhone。

·初学者误区：①以为 iPhone 上不能学 Python（能，用应用就行）；②以为 iOS 和 Android 的 Python"体验一样"（iOS 沙箱严格得多，共享文件靠"文件"App 中转）；③指望 iOS 的 Python 能自由读写手机文件（那是平台不许的）。

代码分析

本节书中没有给出命令行——iOS 上的 Python 全部发生在应用内部（Pythonista 3 / Pyto 的交互式会话与编辑器

GUI 中)。交互式会话、编辑-运行、文件共享这三件事与 PC 上的 IDLE 思路一致，只是载体变成了触屏 App。

A.7 独立应用与可执行程序 (Standalone Apps and Executables)

A.7.1 概念：把 Python 打包成普通程序 (Concept)

英文原文

Besides running Python source code with the traditional schemes we've just met, it's also possible to bundle Python code into a standalone program that users run the same way they run any other program on their device (e.g., by a click or tap). In fact, users can't even tell these bundles are written in Python at all: no source code is visible, no other installs or apps are required, and changes in a locally installed Python have no effect on the bundle. The way you'll build standalones varies per platform. As a noncomprehensive sample of prominent tools today, you can build standalone executables for Windows and Linux with PyInstaller; standalone apps for macOS with PyInstaller and py2app; and standalone apps for Android and iOS with Buildozer and Briefcase (the latter also offers options for PCs).

中文翻译

除了用我们刚见过的传统方式运行 Python 源代码之外，还可以把 Python 代码打包成独立程序（standalone program），让用户像运行设备上任何其他程序一样运行它（例如点击或轻触）。事实上，用户根本看不出这些程序是用 Python 写的：看不到任何源代码，不需要安装其他东西或应用，本地安装的 Python 发生任何变化也不影响这个程序。构建独立程序的方式因平台而异。作为当下众多主流工具的抽样（远非全部）：可以用 PyInstaller 为 Windows 和 Linux 构建独立可执行文件；用 PyInstaller 和 py2app 为 macOS 构建独立应用；用 Buildozer 和 Briefcase 为 Android 和 iOS 构建独立应用（后者也提供 PC 选项）。

深度理解

- 核心概念：独立程序 = "源代码 + 解释器 + 依赖库" 整体打包成一个可交付物。用户不需要 Python，甚至不知道有 Python。
- 底层实现：以 PyInstaller 为例：它会分析脚本的 import 图，把解释器（pythonXX.dll/so）、字节码、标准库和第三方库收集进一个目录或单个可执行文件，启动时在一个临时目录里自解压运行。所以"本地 Python 变化不影响 bundle"——它运行的是自己肚子里的那份 Python。
- 设计思想：这是 Python 可嵌入性（embeddable）的终极形态：语言退居幕后，产品走上前台。对分发软件给非程序员用户是刚需。
- 实际问题：各平台工具分化为"PC 系（PyInstaller/py2a

pp) "和"移动系 (Buildozer/Briefcase) ", 因为移动端的打包涉及 APK/IPA 签名、权限声明等完全不同的问题。

·初学者误区：以为独立打包能"加速"程序（不能，解释器照跑）；以为打包是学习 Python 的早期必学技能（作者明确说：新手阶段完全用不上，后面讲了理由）。

A.7.2 案例：作者自己的跨平台 App (A Cross-Platform App Demo)

英文原文

On Android, for instance, it's possible to develop standalone apps completely in Python. Although this takes more effort than running code in another app, its products are fully functional and idiomatic GUI apps coded in Python, which run with a tap, leverage Android APIs when needed, and can be both side-loaded and uploaded to app stores. As a demo, Figure A-16 captures one of many Python-coded standalone apps for Android, running on a foldable. This app, made by this book's author, is freely available in the Play store; is built for Android with Buildozer; uses the portable Kivy toolkit for its GUI; and relies on Kivy's pyjnius to access Android Java APIs when required in a small fraction of its code (e.g., to request permissions, run services, open docs, and get drive labels). Crucially,

such apps can also run on PC platforms—Windows, macOS, and Linux—from the same code base. Figure A-17, for instance, shows the same app running on macOS. Its code is bundled for macOS and other PCs with PyInstaller; its Kivy GUI is automatically cross-platform; and its POSIX file-sync code works everywhere. The net result is a Python-coded app that runs across a range of PC and mobile hosts, with native behavior on each. To see this for yourself, fetch this app's Android version on Play and its PC versions at this book's website or quixotely.com. Disclaimer: if the Android app is unavailable on Play, check for it at the latter two sites or try a web search; book lifespans tend to be substantially longer than those of apps dependent on stores and platforms (see Termux's troubles!).

中文翻译

以 Android 为例，完全用 Python 开发独立应用是可行的。虽然这比在别的应用里运行代码费劲得多，但产出是功能完整、地地道道的 Python 编写的 GUI 应用：轻触即运行，需要时调用 Android API，既可以被侧载（side-load），也可以上传到应用商店。作为演示，图 A-16 抓拍了众多 Python 编写的 Android 独立应用之一，运行在一台折叠屏设备上。这个应用出自本书作者之手，在 Play 商店免费提供；它用 Buildozer 为 Android 构建；用可移植的 Kivy 工具包做 GUI；在其代码的一小部分中，依靠 Kivy 的 pyjnius 在需要时访问 Android 的 Java API（例如请求权限、运行服务、打开文档、获取驱动器标签）。关键的是，这类

应用还能基于同一份代码跑在 PC 平台上——Windows、macOS 和 Linux。例如，图 A-17 展示了同一个应用运行在 macOS 上。它的代码用 PyInstaller 为 macOS 和其他 PC 打包；Kivy GUI 自动跨平台；POSIX 文件同步代码到处都能跑。最终成果是：一个 Python 编写的应用，跑在一系列 PC 和移动主机上，在每个平台都有原生表现。想亲眼看看，请在 Play 上获取这个应用的 Android 版，在本书网站或 quixotely.com 获取它的 PC 版。免责声明：如果 Android 版在 Play 上不可用，请去后两个网站看看，或试试网络搜索；书的寿命往往远长于那些依赖商店和平台的应用（想想 Termux 的遭遇！）。

深度理解

·核心概念：这是本章的“高光时刻”——一份 Python 代码，Android/macOS/Windows/Linux 通吃。作者用自己的真实 App 证明：可移植性是 Python 的骨血，而不只是宣传语。

·底层实现：App 的组成清晰分层：Kivy 提供跨平台 GUI（它用 OpenGL 自绘界面，不依赖任何原生控件）；pyjnius 是“桥”——用 Python 调 Java API (JNI 技术)；Buildozer/PyInstaller 是“打包器”。语言、GUI、系统桥、打包器各司其职。

·设计思想：作者刻意选“文件同步”这种 POSIX 味十足的功能——文件操作在三大桌面平台和 Android 上行为一致，正好展示“同一份代码”的含金量。反例则是 iOS（没有用户可访问的文件系统，后面提到）。

·实际问题：作者连免责声明都写好了——商店与平台政策会变，书里的链接可能失效。这是对“平台依赖”最现实的提醒。

·初学者误区：①以为 Kivy 是“Android 专用”（它是跨平台的）；②以为“原生表现”= 每个平台的原生界面（Kivy 是自绘界面，风格统一但并非系统原生控件）；③看完本节就想立刻学打包（作者马上会泼冷水：新手阶段无意义）。

A.7.3 其他工具：BeeWare 与关键启示 (BeeWare and the Takeaway)

英文原文

Nor is this toolset the only interoperability game in town. The alternative BeeWare, with its portable Toga GUI toolkit and Briefcase app builder, promises similar platform independence and advertises additional packaging options on PCs. Moreover, some apps built with such tools can work on iOS, too, though its lack of a user-accessible filesystem renders much cross-platform code unusable (e.g., POSIX file-path syncs are impossible). The takeaway here: with a portable programming tool like Python, you're not locked into a single platform's proprietary realm—unless, that is, you develop for platforms that disqualify code that runs anywhere else. As always, choose your coding battles

wisely. Security counts, but closed platforms enable monopolies and stifle innovation. Standalones may not be very useful when you're just getting started with Python (and make no sense at all for running the examples in this book!), but they may become more important when you start writing programs for others to use. When you're ready to explore standalone deliverables in Python, see the web for current tools and details in this domain.

中文翻译

这套工具也不是唯一的互操作方案。另一个选择 BeeWare，以其可移植的 Toga GUI 工具包和 Briefcase 应用构建器，承诺了类似的平台独立性，并在 PC 上宣传了额外的打包选项。此外，用这类工具构建的一些应用也可以在 iOS 上运行，只是 iOS 缺乏用户可访问的文件系统，使大量跨平台代码无法使用（例如 POSIX 文件路径同步根本不可能）。这里的启示是：有了 Python 这样可移植的编程工具，你就不必被锁在单一平台的专有王国里——除非，你选择为那些“否决一切其他地方能跑的代码”的平台开发。一如既往，请明智地选择你的编码战场。安全很重要，但封闭平台助长垄断、扼杀创新。独立程序在你刚开始学 Python 时可能没什么用（对运行本书的示例来说更是毫无意义！），但当你开始为别人写程序时，它们会变得越来越重要。当你有意探索 Python 的独立交付物时，请上网查查这个领域的最新工具和细节。

深度理解

- 核心概念：独立应用生态有两大阵营——Kivy 系 (Buildozer/Kivy/pyjnius) 与 BeeWare 系 (Toga/Briefcase)。它们共同的目标都是"一次编写，处处打包"。
- 底层实现：Toga 与 Kivy 路线不同：Toga 尽量调用平台原生控件（用平台的 UI 框架画按钮），Kivy 则是自绘界面。所以"原生表现"的实现方式也有两种哲学。
- 设计思想：作者在最后给出了全章最有态度的一段话："封闭平台助长垄断、扼杀创新"——这不是技术讨论，而是平台哲学宣言。Python 的意义在于把选择权还给开发者。
- 实际问题：iOS 是跨平台的"阿喀琉斯之踵"——沙箱没有用户级文件系统，任何依赖文件路径同步的代码直接出局。这解释了为什么"跨平台"往往要说"跨了除 iOS 之外的所有平台"。
- 初学者误区：①以为学了 Kivy/BeeWare 就能接活做 App（工具只是最后一公里，前面还有 Python 本身这座山）；②把作者的技术建议当平台站队（作者是"可用性优先"，不是"反苹果"）；③现在就去研究打包——先学语言，打包是"写给用户"阶段的事。

A.8 其他 (Etcetera)

英文原文

While the platform techniques we've explored here are perhaps the simplest and most common ways to use Python, there's much more to this story. For instance, this appendix hasn't said anything about using Python in: - Other IDEs like PyCharm, PyDev, Wing, and VSCode - Web-based notebooks like IPython and Jupyter - Alternative Python implementations like PyPy, Cython, Numba, and Jython - Alternative Python distributions like Anaconda and ActiveState - The cells and macros of spreadsheets like Excel - Web servers using frameworks like Flask and Django - Web browsers using the emerging WebAssembly and Pyodide And lots of other options in no way judged by omission here. This book visits some of these in Chapter 1, summarizes Python implementations in Chapter 2, briefly reviews Jupyter and WebAssembly in Chapter 3, and uses PyPy for benchmarks in Chapter 21. In general, though, advanced usage contexts like these are interesting but out of scope for this Python fundamentals text, and best deferred until you've mastered the language itself.

In the end, Python usage details and options tend to evolve as rapidly as Python itself. Indeed, each prior edition of this book has had to revise its usage coverage radically, and this one expects to fare no better. As noted at the start of this appendix, you should ex

pect to check both Python's docs and the web at large for new-and-exciting developments almost certain to emerge by the time you read these words.

中文翻译

虽然我们在这里探讨的平台技巧也许是使用 Python 最简单、最常见的几种方式，但这个故事的篇幅远不止于此。例如，本附录还没提到在这些场景中使用 Python：- 其他 IDE，如 PyCharm、PyDev、Wing 和 VSCode- 基于 Web 的笔记本（notebook），如 IPython 和 Jupyter- 其他 Python 实现，如 PyPy、Cython、Numba 和 Jython- 其他 Python 发行版，如 Anaconda 和 ActiveState- 像 Excel 这样的电子表格中的单元格和宏- 使用 Flask 和 Django 等框架的 Web 服务器- 使用方兴未艾的 WebAssembly 和 Pyodide 的 Web 浏览器以及很多其他选项——此处列出与否绝不构成任何评判。本书在第 1 章接触过其中一些，在第 2 章总结了 Python 的实现，在第 3 章简要回顾了 Jupyter 和 WebAssembly，并在第 21 章用 PyPy 做基准测试。不过总的来说，这类高级使用场景虽然有趣，但对于这本 Python 基础教材来说超出范围，最好等你自己精通了这门语言之后再涉足。归根结底，Python 的使用细节和选项往往和 Python 本身一样快速演变。事实上，本书的每一个先前版本都不得不彻底修订其“使用篇”的内容，这一版估计也好不到哪去。正如本附录开头所说，你应该做好准备，同时查阅 Python 文档和网上各处，去跟进几乎肯定会在你读到这些文字时冒出来的新进展。

深度理解

·核心概念：本章只讲了"最简单、最常见"的路径。真正的 Python 生态是七条大河：IDE、notebook、其他实现、其他发行版、办公软件、Web 框架、浏览器。

·底层视角：这七个方向背后是 Python 的两种可嵌入形态：解释器嵌入（IDE、Excel 宏、Web 服务器）与替代实现（PyPy 的 JIT、Cython 的 C 化、Jython 的 JVM 化）——它们让 Python 既是一个"语言"又是一种"宿主环境"。

·设计思想：作者把"高级使用场景"全部划到"学完语言之后"——这不是轻视，而是优先级管理：先学会走路（语言核心），再选择交通工具（使用场景）。

·实际问题：WebAssembly/Pyodide 是当下最"未来"的方向——让 Python 在浏览器里跑，可能改变"Python 必须装在本机"的常识。

·初学者误区：①被生态地图吓到（只需知道存在，不需要掌握）；②以为"必须选一个 IDE"才算入门（书里的例子用命令行 + IDLE 就够）；③把"读到的工具清单"当成"要学的清单"——这份清单的正确用法是"需要时再查"。

附录总结

技术要点回顾 (Technical Summary)

把全附录浓缩成一张"平台 × 要点"地图：

平台	推荐路线	核心命令	退出 REP	特色工具	最大的坑
Windows	python.org 安装器 (勾选 PATH 与长	py、python 3.12、python script.py	Ctrl+Z + Enter	py 启动器、IDLE	python3 被商店劫持、点击运行窗口
macOS	python.org 安装器 (Universal 2)	python3、python 3 script.py	Control+D	Python Launcher、IDLE、Homebrew	老机 python 是 2.7、Xcode 弹窗
Linux	预装或 sudo apt install python3	python3、python 3 script.py	Ctrl+D	Gedit、vim/nano、IDLE	tkinter 需单独装 python3
Android	Termux (F-Droid 装, pkg install)	python、python script.py	Ctrl+D (CTRL 按钮)	Termux、Pydroid 3、QuickEdit	Play 版废弃、存储沙箱限制
iOS	App Store 的 Pythonista 3 / Pyto	(App 内操作)	—	Pythonista 3、Toga	沙箱最严、无用户文件系统

跨平台的通用规律：

- 命令三兄弟：python (别名)、python3 (Unix 惯例)、py (Windows launcher) ——三条命令指向同一门语言，但各自平台的"默认入口"不同。
- REPL 退出键：Windows 用 Ctrl+Z，Unix 系 (macOS/Linux/Termux) 用 Ctrl+D——一个按键差异，跨平台新手的第一道坎。
- shebang (#!) 行：Windows 由 py launcher 解析，Unix 由内核/shell 解析——殊途同归，都是"指定用哪个 Python 跑这个文件"。
- 环境变量：PATH (找程序)、PYTHONPATH (找模块)、PYTHONUTF8/PYTHONIOENCODING (定编码)

——三套旋钮，Windows 在系统设置里改，Unix 在 shell 启动文件里改。

·独立打包：PyInstaller (PC)、py2app (macOS)、Briar (Android)、Briefcase (BeeWare 系) ——把"源代码 + 解释器 + 库"整体打包成用户无感的普通程序。

·哲学主线：作者反复强调"保持简单" (keeping it simple) ——每个平台只推荐一条最短路径；同时反复提醒"这是快照"——查官方文档永远比背这本书靠谱。

学习建议 (Learning Advice)

·重要程度：★★★☆☆ (3/5)。它不教任何语法，但决定了你能不能"跑起来"。语法学得再好，环境装不对，一切为零；然而它的内容又高度依赖平台，学完即忘也正常——这是一章"按需查阅"的章节，不是"通读背诵"的章节。

·应该掌握到什么程度：

·必会（二选一即可）：在你自己的电脑上把 Python 装好并跑通 `print('Hello')` (Windows 用 `py`, macOS/Linux 用 `python3`)；会用 IDLE 或任意编辑器运行一个脚本文件；会退出 REPL。

·必须理解：`py` vs `python` vs `python3` 的来龙去脉；`PAT`H/`PYTHON`PATH 是什么、错在哪；"点击运行输出会丢"这件事背后的原理；`shebang` 行在 Windows 与 Unix 上的两种处理方式。

·了解即可：WSL、Cygwin、Homebrew、Termux、Py

droid 3、Kivy、PyInstaller 等——知道"有这个东西、大概解决什么问题"就够，用到时再查。

·不要要求：记住各平台的每条命令；学会从源码编译；现在就去研究独立打包。

·后续学什么：

·立即：第 2 章（Python 如何运行程序——字节码、解释器）与第 3 章（如何运行程序——交互模式、脚本、流重定向、-m 的完整原理）。本附录里的很多命令（如 -q、> 输出重定向、-m）在第 3 章会得到系统性解释。

·近程：第 4 章起正式进入语言核心；学到第 22 章时回头再看 PYTHONPATH 的示例，会豁然开朗。

·进阶（学完本书后）：按 A.8 的生态地图选方向——Web（Flask/Django）、数据（Jupyter/Anaconda）、性能（PyPy/Cython/Numba）或分发（PyInstaller/Kivy）。

·实践建议：安装时务必勾选"Add Python to PATH"；装好后花 5 分钟把 py -3.12（或 python3）敲熟；如果你将来要写给别人用的工具，记住本附录末尾的一句话——独立打包是"交付阶段"的技能，不是"学习阶段"的技能。

附录完。下一站：正文第 2 章 How Python Runs Programs（Python 如何运行程序）。